

DEVELOPMENT OF A DIGITAL
TROUBLESHOOTING GUIDE SYSTEM (TGS)
TO ENHANCE THE EFFICIENCY OF RAILWAY
OPERATIONS BY PROVIDING SYSTEMATIC
GUIDANCE AND SOLUTIONS DURING
INCIDENT MANAGEMENT

NUR NABILAH BINTI SUZELAN AMIR

BACHELOR OF APPLIED SCIENCE IN DATA
ANALYTICS WITH HONOURS
UNIVERSITI MALAYSIA PAHANG AL-
SULTAN ABDULLAH

UNIVERSITI MALAYSIA PAHANG AL-SULTAN ABDULLAH

DECLARATION OF THESIS AND COPYRIGHT

Author's Full Name : NUR NABILAH BINTI SUZELAN AMIR
Date of Birth : 25 AUGUST 2003
Title : DEVELOPMENT OF A DIGITAL TROUBLESHOOTING
GUIDE SYSTEM (TGS) TO ENHANCE THE
EFFICIENCY OF RAILWAY OPERATIONS BY
PROVIDING SYSTEMATIC GUIDANCE AND
SOLUTIONS DURING INCIDENT MANAGEMENT
Academic Session : 2024/2025

I declare that this thesis is classified as:

- ☐ CONFIDENTIAL (Contains confidential information under the Official Secret Act 1997)*
☒ RESTRICTED (Contains restricted information as specified by the organization where research was done)*
☐ OPEN ACCESS I agree that my thesis to be published as online open access (Full Text)

I acknowledge that Universiti Malaysia Pahang Al-Sultan Abdullah reserves the following rights:

1. The Thesis is the Property of Universiti Malaysia Pahang Al-Sultan Abdullah
2. The Library of Universiti Malaysia Pahang Al-Sultan Abdullah has the right to make copies of the thesis for the purpose of research only.
3. The Library has the right to make copies of the thesis for academic exchange.

Certified by:



(Student's Signature)

030825-14-0276
Date: 22nd July 2025



(Academic Tutor's Signature)
DR. SAHIMEL AZWAL BIN
SULAIMAN
Date: 22nd July 2025



(Industry Coach's Signature)

TS. MUHAMMAD SHAFIQ ILHAM BIN
BAHAROM
Date: 22nd July 2025

NOTE : * If the thesis is CONFIDENTIAL or RESTRICTED, please attach a thesis declaration letter.

THESIS DECLARATION LETTER (OPTIONAL)

Librarian,
Perpustakaan Universiti Malaysia Pahang Al-Sultan Abdullah,
Universiti Malaysia Pahang Al-Sultan Abdullah,
Lebuh Persiaran Tun Khalil Yaakob,
26300, Kuantan, Pahang, Malaysia.

Dear Sir,

CLASSIFICATION OF THESIS AS RESTRICTED

Please be informed that the following thesis is classified as RESTRICTED for a period of three (3) years from the date of this letter. The reasons for this classification are as listed below.

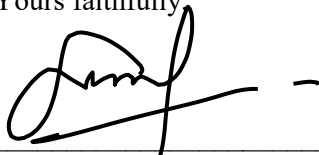
Author's Name Nur Nabilah Binti Suzelan Amir

Thesis Title Development of a Digital Troubleshooting Guide System (TGS) to Enhance the Efficiency of Railway Operations by Providing Systematic Guidance and Solutions During Incident Management.

- | | | |
|---------|------|---|
| Reasons | (i) | The research involves data that is considered sensitive or proprietary. |
| | (ii) | Collaborations with industry partners include Non-Disclosure Agreements (NDAs) that restrict the publication and dissemination of certain data. |

Thank you.

Yours faithfully,



(Academic Tutor's Signature)

Date: 22nd July 2025

Stamp:

Note: This letter should be written by the supervisor, addressed to the Librarian, *Perpustakaan Universiti Malaysia Pahang Al-Sultan Abdullah*, with its copy attached to the thesis.



SUPERVISOR'S DECLARATION

We hereby declare that we have checked this project report, and in our opinion, this final report of Data Science Project is adequate in terms of scope and quality for the award of the Bachelor of Applied Science in Data Analytics with Honours.

(Academic Tutor's Signature)

Full Name : DR. SAHIMEL AZWAL BIN SULAIMAN

Position : SENIOR LECTURER UMPSA

Date : 22nd July 2025

(Industry Coach's Signature)

Full Name : TS. MUHAMMAD SHAFIQ ILHAM BIN BAHAROM

Position : SENIOR ENGINEER GROUP SYSTEM ASSURANCE

Date : 22nd July 2025



**UNIVERSITI MALAYSIA PAHANG
AL-SULTAN ABDULLAH**

STUDENT'S DECLARATION

I hereby declare that the work in this project report is based on my original work.

A handwritten signature in black ink, consisting of a stylized 'N' and 'A'.

(Student's Signature)

Full Name : NUR NABILAH BINTI SUZELAN AMIR

ID Number : SD22053

Date : 22nd July 2025

DEVELOPMENT OF A DIGITAL TROUBLESHOOTING GUIDE
SYSTEM (TGS) TO ENHANCE THE EFFICIENCY OF RAILWAY
OPERATIONS BY PROVIDING SYSTEMATIC GUIDANCE AND
SOLUTIONS DURING INCIDENT MANAGEMENT

NUR NABILAH BINTI SUZELAN AMIR

Data Science Project Report submitted in fulfilment of the requirements
for the award of the degree of
Bachelor of Applied Science in Data Analytics with Honours

Centre for Mathematical Sciences
UNIVERSITI MALAYSIA PAHANG AL-SULTAN ABDULLAH

JULY 2025

ACKNOWLEDGEMENT

First and foremost, I would like to express my deepest gratitude to Allah SWT for His endless blessings, strength, and guidance throughout my academic journey and the completion of this project. Without His will and mercy, none of this would have been possible.

I would like to express my heartfelt gratitude to all the lecturers who have guided me since the beginning of my academic journey in semester one. Your dedication, encouragement, and invaluable knowledge have greatly shaped my understanding and growth in this field. A special thank you to my Academic Tutor (AT), Dr. Sahimel Azwal Bin Sulaiman, and my Industry Coach (IC), En. Shafiq Ilham Bin Baharom, for their unwavering support, insightful feedback, and patience throughout the course of this project.

To my parents and family, your unwavering support, sacrifices, and constant encouragement have been the foundation of my academic journey. Your belief in me has given me the strength and determination to persevere and succeed in my endeavors.

To my friends and colleagues, thank you for your companionship, helpful suggestions, and engaging discussions that enriched my learning experience. Your support and camaraderie have been invaluable during challenging times, and I am truly grateful for your presence throughout this journey.

Lastly, I would like to extend my heartfelt appreciation to myself for the dedication, perseverance, and strength I have shown throughout this journey. Through every challenge, late night, and moment of doubt, I kept going. I am truly proud of how far I have come, the lessons I have learned, and the growth I have experienced along the way. May this final chapter of my degree conclude beautifully, and may it pave the way for a meaningful and rewarding career in the future. With hope, gratitude, and a determined heart, I look forward to the next chapter with confidence and joy.

ABSTRAK

Projek ini membangunkan Sistem Panduan Penyelesaian Masalah Digital (TGS) yang bertujuan untuk meningkatkan kecekapan operasi perkhidmatan rel Prasarana. Pada masa ini, proses penyelesaian masalah tidak disokong oleh penyelesaian automatik secara masa nyata dan masih bergantung kepada rujukan manual. Kebergantungan ini menyebabkan ketidakcekapan, terutamanya ketika insiden kritikal yang memerlukan tindakan segera dan keputusan yang tepat. Selain itu, pendekatan sedia ada sangat bergantung kepada kepakaran pegawai operasi atau pegawai eksekutif, menjadikan kakitangan kurang berpengalaman sukar untuk menyelesaikan masalah secara efektif. Bagi mengatasi keadaan ini, projek ini menggabungkan Pemprosesan Bahasa Semula Jadi (NLP) bersama proses Ekstrak, Transformasi dan Muat Naik (ETL) untuk membersihkan dan menyediakan data tidak berstruktur bagi tujuan pemadanan masalah dan pengecaman punca utama dengan lebih tepat. Sistem ini dibina menggunakan model hibrid yang menggabungkan Frekuensi Istilah-Frekuensi Songsang Dokumen (TF-IDF) dan Perwakilan Penyandi Dua Hala Ayat daripada Transformer (SBERT). TF-IDF berfungsi berdasarkan kata kunci, manakala SBERT memahami makna ayat secara semantik, membolehkan cadangan penyelesaian yang lebih relevan. Sistem turut dilengkapi dengan Antara Muka Pengguna Grafik (GUI) interaktif menggunakan Streamlit, Python dan CSS untuk memudahkan penggunaannya oleh kakitangan operasi. Secara keseluruhan, projek ini membantu mengurangkan masa henti, mempercepatkan keputusan penyelenggaraan, serta menyokong visi Prasarana ke arah mobiliti bandar yang lebih efisien dan selari dengan teknologi Revolusi Industri 4.0 dalam sektor pengangkutan awam Malaysia.

ABSTRACT

This project presents the development of a digital Troubleshooting Guide System (TGS) aimed at enhancing the operational efficiency of Prasarana's railway services. At present, the troubleshooting process lacks a real-time, automated solution and relies on manual references. This reliance leads to inefficiencies, especially during critical incidents where swift and accurate decisions are vital. Furthermore, the current approach is significantly reliant on the expertise of operation controllers or executive operation personnel making it difficult for less experienced personnel to respond effectively in resolving issues effectively. To address these limitations, the project integrates Natural Language Processing (NLP) with an Extract, Transform, Load (ETL) pipeline to clean and prepare unstructured data for accurate issue matching and root cause identification. A hybrid model combining Term Frequency-Inverse Document Frequency (TF-IDF) and Sentence Bidirectional Encoder Representations from Transformers (SBERT) is employed to enhance the relevance and accuracy of recommendations. The TF-IDF component facilitates keyword-based matching, while SBERT enables semantic understanding of user input, allowing the system to suggest the most appropriate solutions efficiently. A user-friendly and interactive Graphical User Interface (GUI), developed using Streamlit with Python and CSS, ensures accessibility for operational staff. Overall, this system aims to reduce downtime, improve maintenance decision-making, and support Prasarana's mission of advancing mobility and enriching experiences. It also aligns with their vision of becoming the first choice of urban mobility, contributing to the broader adoption of Industry Revolution 4.0 technologies within Malaysia's public transport sector.

TABLE OF CONTENTS

TITLE PAGE	
DECLARATION	
ACKNOWLEDGEMENTS	i
ABSTRAK	ii
ABSTRACT	iii
TABLE OF CONTENT	iv
LIST OF TABLES	vii
LIST OF FIGURES	ix
LIST OF ABBREVIATIONS	x
LIST OF APPENDICES	xi
CHAPTER 1 INTRODUCTION	1
1.1 Research Background	1
1.2 Problem Statement	2
1.3 Research Questions	3
1.4 Research Objectives	3
1.5 Research Scopes	4
1.6 Significance of Study	4
1.7 Conclusion	5
CHAPTER 2 LITERATURE REVIEW	6

2.1	Introduction	6
2.2	Digital Troubleshooting Guide System (TGS)	6
2.3	Data Preprocessing Techniques for Unstructured Data	11
2.3.1	Preprocessing Using Natural Language Processing (NLP)	11
2.3.2	Preprocessing Using Optical Character Recognition (OCR)	12
2.3.3	Preprocessing Using Computer Vision (CV)	12
2.4	Pipeline Architecture in Developing System	16
2.5	Integration of Artificial Intelligent Methods for Developing Digital TGS	24
2.6	Models for TGS Development	29
2.6.1	Sentence Bidirectional Encoder Representations from Transformers (SBERT)	29
2.6.2	Term Frequency-Inverse Document Frequency (TF-IDF)	31
2.7	Model Evaluation	35
2.8	Conclusion	38
CHAPTER 3 RESEARCH METHODOLOGY		39
3.1	Introduction	39
3.2	Research Design	39
3.3	Business Understanding	42
3.4	Analytical Approach	42
3.5	Data Requirements	42
3.6	Data Collection	43
3.7	Data Understanding	44
3.8	Packages and Libraries Used	45
3.9	Data Preparation	47
3.10	Modelling	47
3.11	Evaluation	49
3.12	ETL Pipeline	50
3.13	Deployment	51
3.14	Feedback	52
3.15	Conclusion	52
CHAPTER 4 DATA ANALYSIS, RESULTS AND DISCUSSION		53

4.1	Introduction	53
4.2	Data Collection	53
4.3	Data Understanding and Exploration	54
4.4	Data Preparation	56
4.4.1	Data Cleaning	56
4.5	Exploratory Data Analysis (EDA)	57
4.6	Hybrid Model Development	61
4.7	Hybrid Model Evaluation	62
4.8	Graphical User Interface (GUI)	63
CHAPTER 5 CONCLUSION AND RECOMMENDATIONS		69
5.1	Introduction	69
5.2	Conclusion	69
5.3	Recommendation	70
5.4	Entrepreneurship Idea	71
REFERENCES		72
APPENDICES		76

LIST OF TABLES

Table 2.1	Findings of Troubleshooting System and Digitalization	9
Table 2.2	Comparison of Preprocessing Techniques for Unstructured Data	14
Table 2.3	Studies on Architecture Pipelines and System Integration	21
Table 2.4	Studies on Integrating AI for Create System	26
Table 2.5	Table of Summary for Modelling	33
Table 2.6	Table of Summary for Model Evaluation	37

LIST OF FIGURES

Figure 2.1	Nissist Automate System Incident Framework	7
Figure 2.2	Dataverse ETL Pipeline Framework for LLM	16
Figure 2.3	Dataverse ETL Pipeline Framework for LLM	17
Figure 2.4	ETL Pipelines in BI System	17
Figure 2.5	GenAI in Automate ETL Pipeline	18
Figure 2.6	Data Modelling in Optimizing Pipeline Architecture	19
Figure 2.7	Digital Reporting System utilizing Data Warehouse	19
Figure 2.8	SBERT Architecture	30
Figure 2.9	Model Comparison Test Results	31
Figure 2.10	SBERT Architecture for Compute Similarity Scores	35
Figure 3.1	Research Framework	39
Figure 3.2	Research Gantt Chart	40
Figure 3.3	Agile Methodology	41
Figure 3.4	Libraries Used	45
Figure 3.5	ETL Pipeline Structure	50
Figure 4.1	Raw Data from Minimum Operation Requirement (MOR)	53
Figure 4.2	Check Duplicate Data	54
Figure 4.3	Check Data Type and Missing Text	54
Figure 4.4	Fill in Missing Text	55
Figure 4.5	Data Cleaning Process	56
Figure 4.6	Frequency of Failure Classification	58
Figure 4.7	Word Cloud for HMI, FSCADA and ATS Alarm Description	58
Figure 4.8	Top 10 of Guidelines for the Chief Controller	59
Figure 4.9	Top 10 of Local Response	60

Figure 4.10	Develop Hybrid Model	61
Figure 4.11	Evaluation using Cosine Similarity	62
Figure 4.12	User Authentication	63
Figure 4.13	Access Denied to Login	63
Figure 4.14	User Interface and Upload Data	64
Figure 4.15	Data Preview	65
Figure 4.16	Selection Area of Concern	65
Figure 4.17	User can Search and Select Failure Scenario	66
Figure 4.18	Model Comparison and Finding Best Match Failure Scenario	67
Figure 4.19	Timer, Alarm Status Triggered and Solutions Recommendations	68
Figure 4.20	Data Logged in a CSV	68

LIST OF ABBREVIATIONS

TGS	Troubleshooting Guidance System
AI	Artificial Intelligent
NLP	Natural Language Processing
OCR	Optical Character Recognition
CV	Computer Vision
ETL	Extract, Transform, Load
IR 4.0	Industrial Revolution 4.0
SBERT	Sentence Bidirectional Encoder Representations from Transformers
TF-IDF	Term Frequency-Inverse Document Frequency
GUI	Graphical User Interface
EDA	Exploratory Data Analysis
KGL	MRT Kajang Line
MOR	Minimum Operating Requirement

LIST OF APPENDICES

Appendix A: DSP2 Full Code	75
Appendix B: Link App after Deployment	75

CHAPTER 1

INTRODUCTION

1.1 Research Background

Transportation sector holds an important role in economic growth and the quality of life through connecting people and goods across regions. Railways play an important role in public transit, especially cities, as they ease road congestion and support dense populations. Malaysia embarked on its railway journey in 1885 with a line linking Taiping to Port Weld in Perak, initially to sustain the tin mining industry (Chan, 2020). Over the years, Malaysia's railways have equally undergone changes, from steam to diesel engines and finally electric trains, with the development of rails globally. Notable achievements include the 1996 debut of the Light Rail Transit (LRT), which revolutionized urban mobility, as well as current projects such as LRT3, Mass Rapid Transit 3 (MRT3), the East Coast Rail Link, and the KL-Singapore High-Speed Rail, which aim to improve connectivity and encourage sustainable development (Chan, 2020).

As the railway infrastructure advances, managing its complexity in operation becomes more important. Expanding infrastructure and increasing ridership requires effective systems to maintain safety and reliability. The adoption of Industry 4.0 (IR 4.0) technologies tools such as data analytics, automation, and artificial intelligence (AI) that provides revolutionary solutions. According to Schwab (2018), tools like these are essential for improving safety and efficiency. These tools help by identifying potential issues before they happen and automating routine tasks. A troubleshooting system is fundamental as it provides an organized approach to identifying, evaluating, and resolving system issues. Similarly to International Business Machines Corporation (IBM) emphasizes that troubleshooting systems allow operators to

systematically investigate and resolve issues, leading to better management. Hence, an important step towards resolving operational issues with Malaysia's railways is the implementation of a digital Troubleshooting Guidance System (TGS) developed with IR 4.0 technologies. TGS improves train service reliability, reduces system downtime, and minimizes human error by utilizing automation, centralized data systems, and predictive analytics.

Moreover, the introduction of the digital TGS is part of Malaysia's bigger policy imperatives reflected in RMK-12, which stipulates how much digital transformation is critical for achieving the more inclusive and sustainable economic development. Through the implementation of advanced troubleshooting companies like Prasarana Malaysia Berhad (Prasarana) will achieve greater satisfaction for passengers, reduced environmental impact through better energy management, and operating efficiency. Thus, this system allows easy sharing information and documentation management to capture insights at the operation level for use in continuous improvement procedures and ensure the insights from day-to-day operations.

Malaysia's railway infrastructures dedicate to build an economical and sustainable public transportation system is demonstrated by the renovation of its railway infrastructure with IR 4.0 technologies such as TGS. This innovation will be beneficial for railways to meet the urbanizations demands while established a standard for digital innovation in public transportation around the world. With strategic planning combined with technology, Malaysia's rail network has been secure, dependable and efficient. Malaysia not only improves its public transport infrastructure but also paves the way as how emerging companies may implement IR 4.0 in support of urban development and environmental and environmental sustainability.

1.2 Problem Statement

Currently, the troubleshooting process in railway operations lacks a real-time problem-solving solution and heavily depends on manual references, particularly the Minimum Operating Requirement (MOR) document. This reliance often results in inefficiencies during the decision-making process, especially in critical situations where time and accuracy are

essential. Moreover, the existing approach depends strongly on human expertise, particularly from experienced Operation Controllers or Executive Operations personnel. As a result, less experienced staff or non-technical personnel may struggle to diagnose and resolve issues promptly, leading to delays, miscommunication, or inconsistent responses in handling equipment failures.

1.3 Research Questions

- How can document-based operational data be ingested, stored, and processed to generate actionable operational insights for Prasarana?
- Which system can be developed to enhance operational efficiency in Prasarana?
- How to develop the GUI that can enables staff to easily access with TGS?

1.4 Research Objectives

- To design an ETL-based architecture pipeline to manage and analyze data for generating operational insights for Prasarana.
- To develop a digital TGS with hybrid model: TF-IDF and SBERT that can enhance Prasarana's operational efficiency by supporting real-time solutions recommendations and decision-making during failure incident management.
- To design a user-friendly and interactive GUI using Streamlit that enables staff to easily access with TGS, simplifying processes, and enhancing both user and customer experience.

1.5 Research Scopes

This project was conducted over the span of one year, from March 2024 to March 2025, and focuses on the development of a TGS to assist Prasarana, specifically for the MRT Kajang Line. The primary data source is the Minimum Operating Requirement (MOR), a standardized operational guideline used continuously by staff without a fixed historical period, as it is only updated when necessary. Data preprocessing and Exploratory Data Analysis (EDA) were performed using Google Colab, while the system's GUI was developed using Streamlit in Visual Studio Code, utilizing Python and CSS. The system enables users to upload relevant data in CSV or XLSX format. All user interactions are automatically logged into a CSV file, creating a historical database for upper-level monitoring, which can also be downloaded for reference. A hybrid model combining TF-IDF and SBERT is implemented to support real-time retrieval of failure scenarios based on user queries. Cosine Similarity is used to evaluate the relevance of the retrieved matches. This system is specifically designed for use by Operational Control Centre (OCC) personnel and technical staff to support consistent decision-making based on the MOR guideline.

1.6 Significance of Study

The system is developed to improve how Prasarana's technical staff and operations teams work by automate the process in managing incidents and providing real-time and solve the problems occurs during the incidents management. Furthermore, the system helps staffs respond faster and accurately by automating the process of managing incidents and providing real-time data.

In addition, the system is vital for Prasarana in enhancing their service and quality in operational efficiency. Accessing all the incident data from a database will be very helpful to improve the way to handle accidents, minimize errors, and make the process more reliable. In addition, integrating modern technologies like AI and data analytics will help Prasarana to always be in the lead in the field of technology, especially in the public transport sector. Hence, this approach will reduce cost and enhance the services for consolidating Prasarana to its vision of a world-class public transport operator.

This system will also be of great benefit to the community since it makes the usage of public transport even safer and more efficient. There are fewer delays that commuters will face in daily travels, hence the journeys will smoother and less stressful. In this way, it eventually contributes to an improved general quality of life and increased public transportation as daily transportation for a more sustainable lifestyle and city. Thus, when the service is improved, it will also boost social mobility by ensuring all the community can easily access essential services, jobs and educational purposes.

Lastly, for Malaysia, a better transportation system is indispensable in social and economic development. This would be helpful in creating an effective and reliable railway industries, which is vital for Malaysia to continue urbanizing. By reducing the delays and service disruptions, it will enhance the overall quality of transportation system and gain passengers' trust in public transportation. This will also benefit the environment in its own small way since reduced emissions mean less traffic and enhanced life qualities for people in the surroundings.

1.7 Conclusion

This chapter begins with an overview of the research background, the problem statement, three research questions with three research objectives. The scope of research is followed by its significance. Next, the research will explore the existing literature on the development of a digital TGS for railway operations on the next chapter.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

In this chapter, we will explore the methodologies and approaches used in previous studies related to develop troubleshooting systems in the transportation sector that focusing on how AI technologies can improve the operational efficiency and reliability in railway operations during incident managements.

2.2 Digital Troubleshooting Guide System (TGS)

The troubleshooting system is one orderly method in finding the problems in the product, system, or process by users or technicians. These are generally included with step-by-step, easy to follow instructions that allow user to find and resolve the problem's root cause. They often include interactive tools, adaptive checklists for specific scenarios, and a deep knowledge base with documentation, frequently asked questions, and technical solutions. These systems are helpful in reducing delays, improve efficiency, and making sure the solutions provided are accurate. These come in various sources such as manuals, online platforms, or interactive help systems, depends on how these organizations or users may prefer or need (Elmira, 2022).

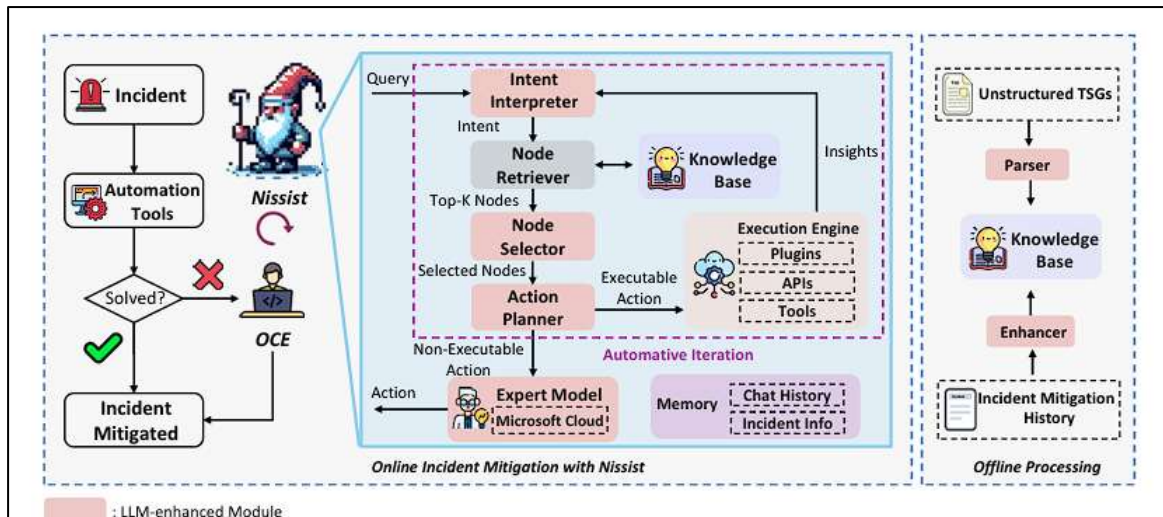


Figure 2.1 Nissist Automate System Incident Framework

Source: An et al. (2024)

As pointed out in Figure 2.1, An et al. (2024) emphasizes that the framework illustrates how a digital tool for troubleshooting can embed expertise, memory modules, and automation. It is designed to work iteratively online or offline for quick and efficient incident resolution with minimal human operators. These flexibilities and automation relate to faster response times and more reliable, which would be applicable for digital TGS in railways sector. In a nutshell, Nissist Automate System and TGS aim at smoothing operations and enhanced real-time data driven solutions without overloading human resources.

Digital transformation gave it totally different dimension. It aims to develop more intelligent, faster and user-friendly troubleshooting systems based on modern technologies such as artificial intelligent, cloud computing, and automation. According to IBM, digital transformation involves the use of data and technology in innovative ways to drive improvements in efficiency and engagement. By combining traditional troubleshooting system offer real-time solutions, automated processes, and a better experience for users. This shift has made troubleshooting systems more effective in handling the demands of today's fast-paced and technology-driven operations.

In the railway industry, digitalization is known as the key driver of future developments. The goal is to provide users with efficient transportation options while use the benefits of digital transformation. According to Tsetkov et al. (2019), digitalization is crucial for railway

management, increase operating efficiency, and develop the responsive systems. Digital tools in railway management enable maintenance operations, real time monitoring, and consistent customer experience. Thus, the digital transformation is vital for sector's growth and competitiveness.

Table 2.1 Findings of Troubleshooting System and Digitalization

No.	Title	Author (s)/ Year	Findings	Limitations
1.	Troubleshooting Guide: Definition, Importance, and Tips for Creating	Elmira (2022)	Describes troubleshooting guides as systems to help diagnose and solve problems quickly, highlighting their role in improving efficiency and reducing downtime.	The article focuses primarily on practical tips for creating troubleshooting guides but lacks empirical data or case studies to validate the effectiveness of the suggested practices.
2.	What is Digital Transformation?	IBM (2024)	Explains how digital transformation, through AI and cloud computing, creates smarter, more efficient systems that improve automation and real-time decision-making.	It assumes a technical understanding of IBM's architecture, making it less accessible for non-technical users

3.	Digital Management Railway	Tsvetkov, V., Shaytura, S., & Ordov, K. (2019)	Stresses that digital transformation is key to meeting the challenges of modern railway systems.	The article does not delve into the specific challenges or limitations of implementing troubleshooting systems in digitally transformed environments.
4.	Nissist: An Incident Mitigation Copilot based on Troubleshooting Guides	An, K., Yang, F., Lu, J., Li, L., Ren, Z., Huang, H., Wang, L., Zhao, P., Kang, Y., Ding, H., Lin, Q., Rajmohan, S., Zhang, D., & Zhang, Q. (2024)	Highlights the potential of AI-driven incident mitigation through the Nissist framework, enhancing troubleshooting guides with expert knowledge and automation to improve incident resolution in real-time.	Potential dependency on high-quality data and advanced infrastructure, which could be challenging for smaller organizations or developing regions.

2.3 Data Preprocessing Techniques for Unstructured Data

According to Sadhvi (2024), data preprocessing refers to changing raw data into an understandable format. These are vital steps for converting the raw, unstructured data to structured, usable insights in systems like digital TGS. Standardization of data and dealing with lost values or outliers will provide correct and consistent data for successful performance improvement and making the system reliable.

2.3.1 Preprocessing Using Natural Language Processing (NLP)

Siino et al. (2023) examined the significance of text preprocessing techniques in modern machine learning models including Transformers and traditional classifiers. The preprocessing methods such as tokenization, lowercasing, and stop-word removal improve the overall performance of these models. Even though, the Transformers does not require preprocessing and accept raw text, preprocessing improves input data quality and speed. Hence, TGS will go through some preprocessing to facilitate the ease of analysis of unstructured incident reports to NLP algorithms.

Chai (2022) conducted a detailed study comparing various preprocessing techniques and their effects on NLP tasks. From the research, it highlights the importance of selecting appropriate methods based on system's objectives. Techniques such as stemming and lemmatization will improve the semantic understanding of text by reducing words to their base forms, which is beneficial particularly in identifying key patterns in incident data. On the other hand, the researcher also warns do not do over preprocessing, since it will delete the valuable context which may be necessary for understanding in complex cases. A balanced preprocessing of data is required to retains its core meaning while being structured for analysis.

The insights from these studies highlight the importance of preprocessing in ensuring the success of the TGS. Techniques like tokenization, stop-word removal, and lemmatization are essential for organizing textual data from incident reports and maintenance logs. By applying these preprocessing steps, the TGS can efficiently analyse incident data, extract

actionable insights, and make accurate troubleshooting recommendations. This structured approach ensures the system operates effectively and supports the goal of improving operational efficiency.

2.3.2 Preprocessing Using Optical Character Recognition (OCR)

Choi and Jeong (2023) researched methods of detecting table objects and text areas in real time as a preprocessing for OCR. That indeed points out that an accurate identification of text and not text regions improve the performances of OCR systems much better. If the preprocessing accurately segregates table structure and text fields, it can remove a lot of irrelevant data and hence cleanse the output quality of OCR. For TGS, these techniques guarantee that structured incident reports and logs are correctly digitalized, preserving critical formatting and content.

Tavares (2024) conducted a comprehensive comparison of image preprocessing techniques in license plate recognition by OCR. According to study, steps like the binarization of images, noise reduction, and contrast adjustment are crucial for enhancing both the performance and accuracy of OCR. While the research focused on vehicle license plates, applicability can be seen in other areas too, such as railway incident records. These can be considered preprocessing techniques in enhancing the clarity of scanned documents for readability also minimize chance of OCR errors.

2.3.3 Preprocessing Using Computer Vision (CV)

Lu et al. (2024) explored a set of preprocessing methods for improved image compression in CV applications. This study identified a few key steps of preprocessing that, if adapted compression really enhances the accuracy of CV and speed. They prepare the visual data for algorithm so that computations can be made quicker, hence can reduce computation overhead.

Tan et al. (2024) discussed the opportunities behind the integration of different advanced techniques of CV and AI algorithms in autonomous driving systems. The researcher mentioned that CV can detect patterns, recognize objects, and analyse real-time visual data. Although the studies focused on autonomous driving system. These techniques may enable the system to interpret the visual incident data, such as identifying equipment failures or hazardous conditions from images or videos.

In a nutshell, both findings highlight the role of CV in handling the visual data. CV may be helpful for TGS if the has figures and want to do detection of visualize incident. This ensures the non-textual data will accurately be processed and integrated into the system. Thus, it will provide insights for decision making.

Table 2.2 Comparison of Preprocessing Techniques for Unstructured Data

No.	Title	Author(s)/Year	Methods	Limitations
1.	Data Preprocessing in Data Mining: A Hands-on Guide	Sadhvi (2024)	Explores data cleaning, transformation, and reduction techniques for preparing data in mining tasks.	Limited applicability to real-time systems or large-scale industrial contexts, as it primarily targets beginners or academic projects
2.	Comparison of Text Preprocessing Methods	Chai (2022)	Compares techniques like stemming, lemmatization, and tokenization for different NLP tasks.	Assumes uniformity in dataset quality, which might not reflect real-world data variations.
3.	Is text preprocessing still worth the time? A comparative survey on the influence of popular preprocessing methods on Transformers and traditional classifiers	Siino, Tinnirello, & La Cascia (2023)	Evaluates the impact of traditional preprocessing methods like stop-word removal on Transformers and classifiers.	Limited exploration of domain-specific impacts, such as its application in troubleshooting systems or OCR-based tasks.
4.	Comparison of Image Preprocessing Techniques for Vehicle License Plate Recognition Using OCR	Tavares (2024)	Assesses image preprocessing methods, including binarization and noise reduction, to enhance OCR accuracy.	Assumes access to high-quality images, which might not be realistic in low-resolution or challenging environmental conditions.

5.	Realtime Detection of Table Objects and Text Areas for OCR Preprocessing	Choi & Jeong (2023)	Proposes real-time text and object detection to improve OCR preprocessing for complex documents.	Focuses on OCR preprocessing but real-time performance results may vary significantly depending on hardware and software environments.
6.	Preprocessing Enhanced Image Compression for Machine Vision	Lu, Ge, Zhong, Hu, & Geng (2024)	Examines preprocessing steps like scaling and noise filtering to optimize image data for machine vision systems.	The study emphasizes compression efficiency and visual quality but does not explore how this affects higher-level tasks like object detection or OCR.
7.	Integrating Advanced Computer Vision and AI Algorithms for Autonomous Driving Systems	Tan, Wu, Zhou, Wang, & Chen (2024)	Combines CV and AI algorithms for object recognition, anomaly detection, and real-time visual data processing.	The study assumes access to advanced computational resources, which may not align with more constrained operational environments.

2.4 Pipeline Architecture in Developing System

According to IBM (2024), a data pipeline is a set of processes and technologies that automates the flow of data from its source to its destination. This includes collecting, transforming, and loading data for analysis or decision-making. A well-structured pipeline is essential for modern systems, as it ensures that data is consistent, accurate, and timely, which is crucial for troubleshooting and incident management in systems like TGS.

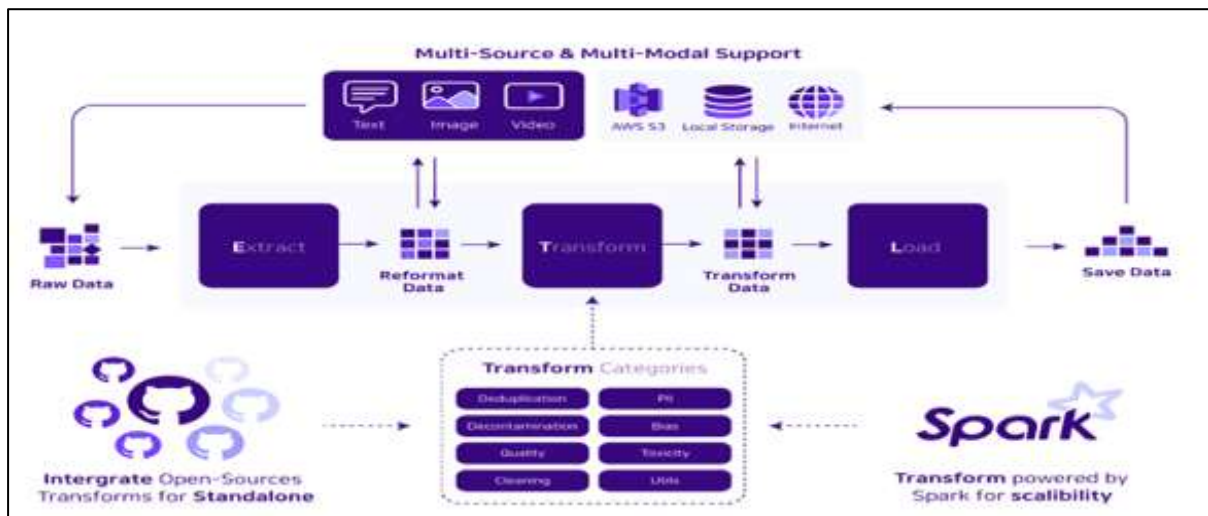


Figure 2.2 Dataverse ETL Pipeline Framework for LLM

Source: Park et al. (2024)

Based on Figure 2.2, Park et al. (2024) introduced Dataverse, an open-source Extract, Transform, Load (ETL) pipeline framework designed for handling data processing for large language models (LLM). The study emphasizes the importance of efficient extraction, transformation, and loading mechanisms to prepare and integrate data for system scalability. The findings highlight that a compatible ETL architecture ensures flexibility and supports real-time updates, which are vital for systems like the TGS to handle continuous incident data input.

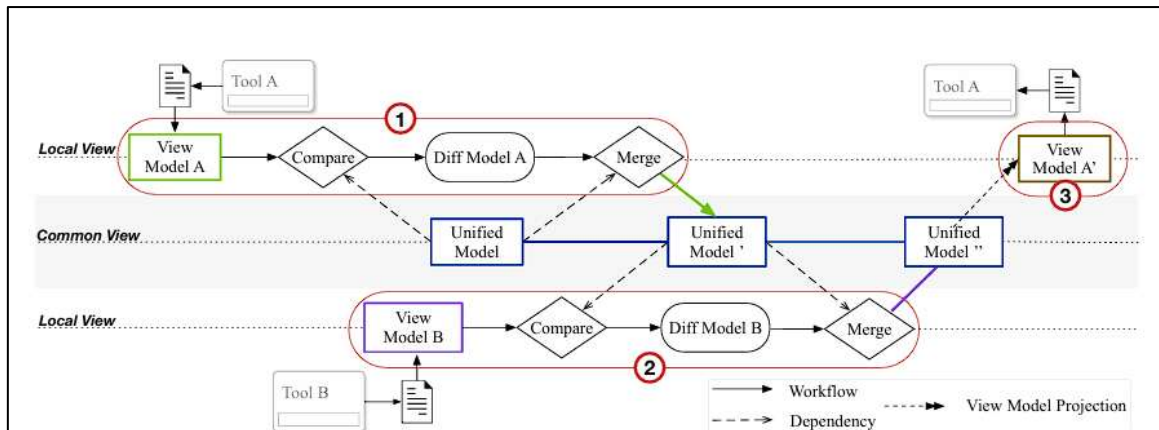


Figure 2.3 Dataverse ETL Pipeline Framework for LLM

Source: Rinker et al. (2021)

Rinker et al. (2021) proposed a model transformation pipeline architecture shown in Figure 2.3 specifically designed for production system engineering. Their work highlights the benefits of continuous integration in managing multi-source data. By creating a strong transformation pipeline, the study demonstrates how data consistency across multiple views improves system design and operational insights. These principles align closely with the TGS, where data from various formats; text, images, logs must be merged into a structured format.

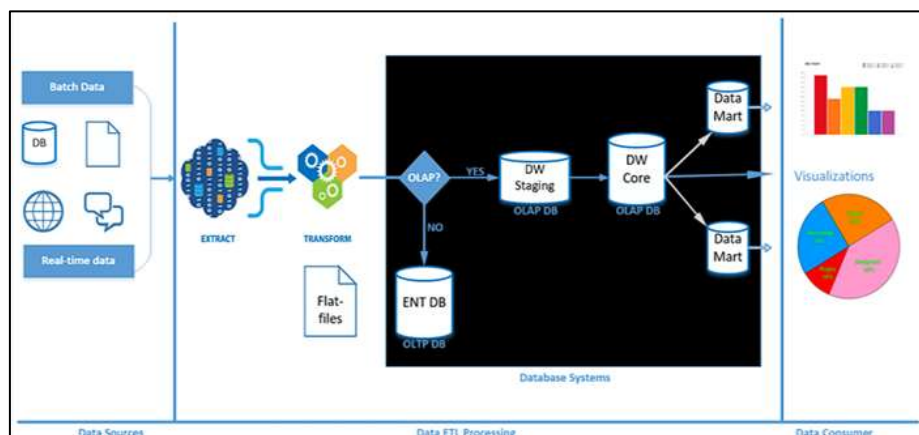


Figure 2.4 ETL Pipelines in BI System

Source: Elom (2023)

Elom (2023) explored the implementation of ETL pipelines for data aggregation and business intelligence (BI) systems as illustrates in Figure 2.4. This research emphasized the importance of data cleaning, normalization, and transformation to ensure the accuracy and usability of data for decision-making. For TGS, similar pipelines can be used to aggregate and analyse historical and real-time incident data, enabling efficient troubleshooting and operational insights.

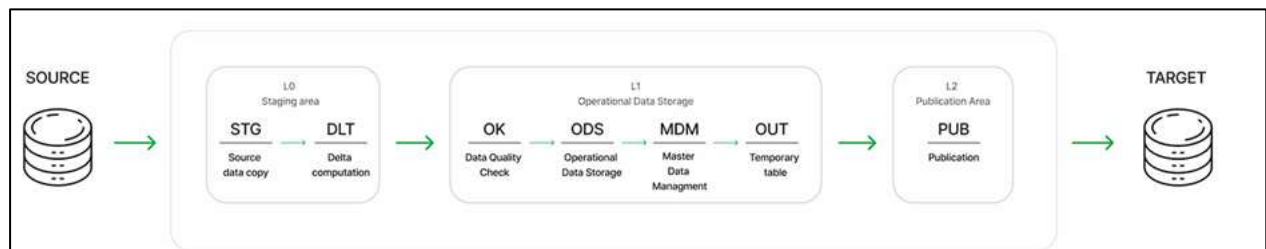


Figure 2.5 GenAI in Automate ETL Pipeline

Source: Putten (2024)

Putten (2024) explored the integration of generative AI (GenAI) in automating ETL pipeline processes as shown in Figure 2.5. The study demonstrated that AI-driven pipelines reduce manual effort, enhance transformation accuracy, and optimize data flow. This approach can be applied to the TGS to streamline the digitization and integration of incident data, making the system more responsive and adaptive.

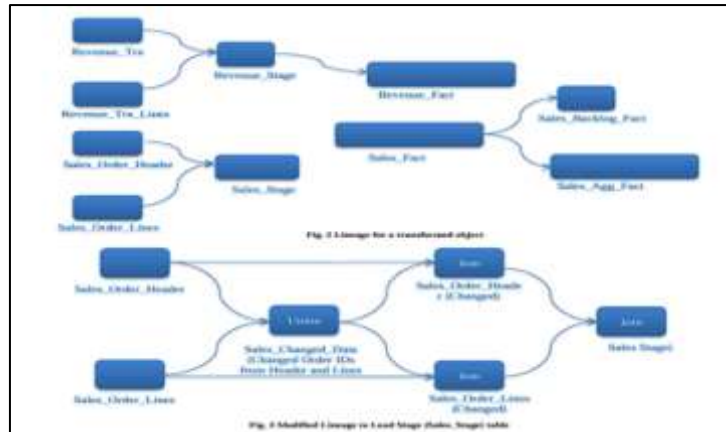


Figure 2.6 Data Modelling in Optimizing Pipeline Architecture

Source: Pandey (2024)

Pandey (2024) emphasized the role of data modelling in optimizing pipeline architecture in Figure 2.6. The research showed that well-structured data models improve the efficiency of pipeline operations by ensuring proper data flow, reducing delay, and enabling effective analysis. For TGS, implementing optimized data models ensures the smooth handling of structured and unstructured data, improving overall system performance.

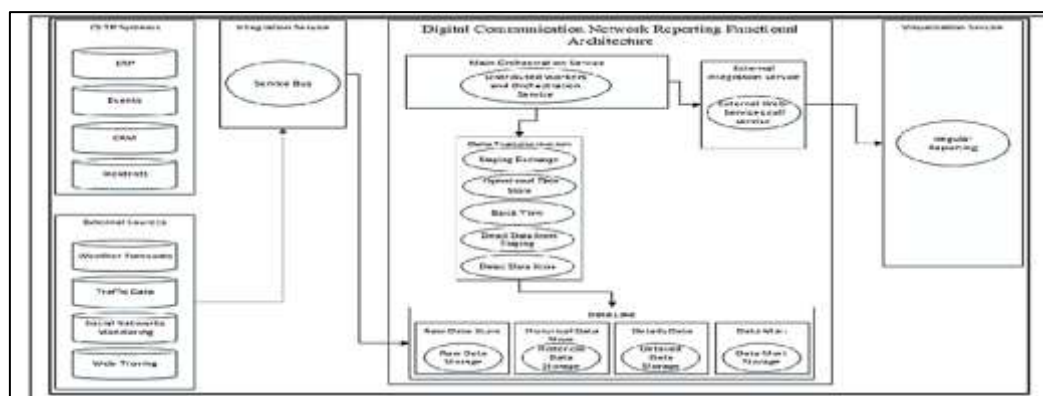


Figure 2.7 Digital Reporting System utilizing Data Warehouse

Source: Suleykin and Panfilov (2022)

Suleykin and Panfilov (2022) presented a case study on a digital reporting system for railway communication networks as shown in Figure 2.7. Their research focused on the importance of automating data flow and integrating centralized reporting pipelines utilizing data warehouse. This framework is directly applicable to the TGS, as it highlights the need for automated processes to streamline incident reporting and improve data accuracy.

Table 2.3 Studies on Architecture Pipelines and System Integration

No.	Title	Author(s)/ Year	Methods	Findings	Limitations
1.	What is a Data Pipeline?	IBM (2024)	Overview of data pipeline processes: extraction, transformation, and loading of data.	Explains the role of data pipelines in automating data flows, improving system integration.	The article provides a general overview of data pipelines without diving deeply into technical implementations or challenges specific to complex systems.
2.	Dataverse: Open-Source ETL (Extract, Transform, Load) Pipeline for Large Language Models	Park et al. (2024)	Open-source ETL pipeline for large-scale data processing with modular architecture.	Highlights scalability and flexibility of ETL pipelines for real-time updates and integration.	Focuses on ETL performance with LLMs but does not explore real-world constraints like resource optimization or scalability in different industries.
3.	Continuous Integration in Multi-view Modeling: A Model Transformation Pipeline Architecture	Rinker et al. (2021)	Multi-view modeling with transformation pipelines for	Improves operational insights by harmonizing multi-source data.	Assumes familiarity with continuous integration (CI) and multi-view modeling, which

			consistent data management.		might not be directly relevant for all ETL pipeline applications.
4.	Implementation of a Data Aggregation ETL Pipeline and Business Intelligence System	Elom (2023)	ETL pipeline for data aggregation, normalization, and transformation for business intelligence.	Enhances data accuracy and usability for decision-making.	Does not consider real-time data processing challenges, which are crucial for dynamic environments like railway communication networks.
5.	Transforming Data Flow: Generative AI in ETL Pipeline Automatization	Putten (2024)	AI-powered ETL pipelines for automated extraction and transformation of data.	AI reduces manual effort and improves efficiency in ETL processes.	Concentrates on using generative AI for ETL automation but lacks comparative analysis with traditional ETL techniques.
6.	Analyze and Optimize Data Pipelines with Effective Data Models	Pandey (2024)	Data modeling to optimize pipeline efficiency and reduce processing bottlenecks.	Optimized data models enhance pipeline performance and efficiency.	The study emphasizes data modeling but provides limited insights into the operational challenges of building and maintaining ETL pipelines.

7.	Railway Communication Network Digital Reporting System: A Case Study	Suleykin & Panfilov (2022)	Automated reporting pipelines for centralized data management in railway communication.	Highlights the need for automated data flow to streamline reporting and improve accuracy.	Focuses on reporting and network communication, with less emphasis on data pipeline performance or scalability challenges.
----	--	----------------------------	---	---	--

2.5 Integration of Artificial Intelligent Methods for Developing Digital TGS

NLP techniques are instrumental in extracting and interpreting meaningful insights from unstructured text data, such as incident reports, maintenance logs, and troubleshooting manuals. As Vajjala et al. (2020) and Herwanto et al. (2024) emphasize, NLP allows systems to classify, tokenize, and summarize textual information, enabling automated and precise incident analysis. By implementing NLP, the TGS can effectively interpret large volumes of unstructured data, facilitating faster and more informed decision-making. NLP's collaborative approach to generating high-quality content, as demonstrated by Lu et al. (2023), highlights its role in providing accurate troubleshooting recommendations and improving operational efficiency.

OCR technology is vital for converting unstructured, paper-based records into machine-readable digital formats, which is critical for modernizing systems like the TGS. Glasenapp et al. (2023) demonstrated OCR's ability to handle irregular documentation effectively, emphasizing its relevance in digitizing historical incident records. Monteiro et al. (2023) extended this application to industrial environments, where OCR combined with object detection enhances accuracy in identifying and processing data from labels and stickers. Charlton (2023) specifically explored OCR's potential in improving documentation processes and integrating data into automated workflows. This study is significant for TGS development, as it shows how OCR can reduce manual effort in handling diverse document types, ensuring more efficient troubleshooting by rapidly digitizing and analysing records. By using such advanced OCR systems, TGS can achieve greater accuracy in data extraction and faster response times during incident management.

CV is a crucial AI tool for interpreting visual data such as images, diagrams, and videos. Venkatesh et al. (2024) introduced MetaPix, a platform designed to manage and utilize unstructured CV data, which is vital for improving decision-making in systems that rely on visual data. In TGS, CV can be used for interpreting images of equipment failures, ensuring that incident data, whether text-based or visual, is processed accurately. Furthermore, Chikhale et al. (2024) explored the application of CV in industrial defect detection, showing how it can

be applied for automatic identification and classification of issues from visual inputs. In TGS, CV can assist in detecting equipment anomalies or safety hazards, providing an additional layer of real-time troubleshooting capabilities.

The combined application of these AI tools results in a highly reliable and safe TGS. NLP ensures that textual data is accurately interpreted, OCR digitizes critical records, and CV processes visual inputs efficiently. Together, these technologies reduce downtime, improve decision-making speed, and enhance the overall safety of railway operations. As Suleykin and Panfilov (2022) illustrate in their case study on railway digital systems, automated pipelines and AI-driven tools significantly enhance reporting accuracy and operational efficiency, setting a strong foundation for reliable incident management systems.

Table 2. 4 Studies on Integrating AI for Create System

No.	Title	Author(s) / Year	Findings	Limitations
1.	Practical Natural Language Processing: A Comprehensive Guide to Building Real-World NLP Systems	Vajjala et al. (2020)	Demonstrates how NLP techniques like text classification and tokenization extract meaningful information from unstructured data.	Primarily focuses on theoretical and practical foundations of NLP but lacks detailed insights into emerging topics like privacy-preserving NLP or real-time deployment.
2.	Leveraging NLP Techniques for Privacy Requirements Engineering in User Stories	Herwanto et al. (2024)	Shows how NLP automates the extraction and interpretation of data in user stories, improving system efficiency.	Focuses on textual user stories, excluding multimedia or other complex data formats and does not explore scalability issues or computational overhead for large-scale datasets.
3.	ReadingQuizMaker: A Human-NLP Collaborative System that Supports Instructors	Lu et al. (2023)	Highlights collaborative use of NLP to generate structured	The application is narrowly focused on quiz generation, relies heavily on human

			content, applicable for automated troubleshooting steps.	intervention, which may reduce scalability for fully automated systems.
4.	OCR Applied for Identification of Vehicles with Irregular Documentation Using IoT	Glaserapp et al. (2023)	Demonstrates OCR's ability to digitize irregular documentation, ensuring accurate and fast data conversion.	Performance may be constrained by varying lighting conditions or hardware capabilities in IoT environments. Does not explore scalability for large-scale deployments across multiple jurisdictions.
5.	A Comprehensive Framework for Industrial Sticker Information Recognition Using Advanced OCR and Object Detection Techniques	Monteiro et al. (2023)	Highlights advanced OCR combined with object detection to improve data accuracy in industrial settings.	Limited analysis of the computational costs associated with integrating OCR and object detection techniques and results may vary significantly across industries with different labeling standards or sticker materials.
6.	OCR Report	Charlton (2023)	Explores how OCR improves document processing and automation workflows for efficient system management.	The report provides a high-level overview of OCR challenges but lacks practical implementation details.

7.	MetaPix: A Data-Centric AI Development Platform for Efficient Management of Unstructured CV Data	Venkatesh et al. (2024)	Introduces CV techniques for managing unstructured visual data, enhancing decision-making capabilities in systems.	Scalability in handling extremely large datasets or real-time processing is not thoroughly evaluated.
----	--	-------------------------	--	---

2.6 Models for TGS Development

2.6.1 Sentence Bidirectional Encoder Representations from Transformers (SBERT)

The need for real-time, intelligent troubleshooting systems has driven the development of NLP techniques that capable of understanding complex queries and retrieving relevant information effectively. SBERT is an advanced sentence embedding technique that modifies the original BERT model by incorporating Siamese and Triplet Networks to generate semantically meaningful vector representations for sentences (Reimers and Gurevych, 2019). While BERT excels in various NLP tasks, it has limitations when applied directly to sentence similarity and semantic search. BERT utilizes a cross-encoder architecture where two sentences are input together, separated by a special token. The model processed both sentences simultaneously to capture their relationship (SentenceTransformers Documentation — Sentence Transformers Documentation, 2019). Reimers and Gurevych (2019) showed that traditional BERT requires significant time and computing power to find similar sentences in a large dataset. For example, comparing 10,000 sentences with BERT requires approximately 65 hours of computation. In contrast, SBERT reduces this to just 5 seconds while achieving equal or better accuracy. This is achieved by feeding sentences independently into two identical BERT models (Siamese structure) and then comparing the resulting embeddings using cosine similarity.

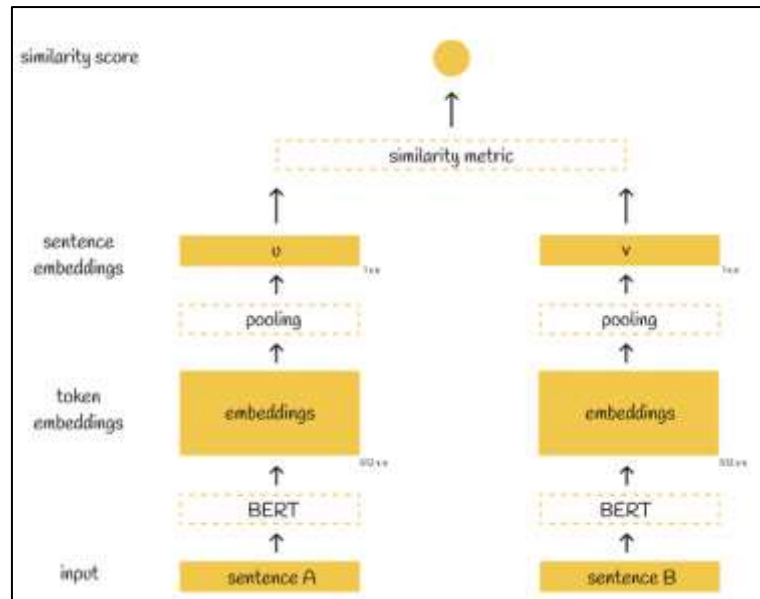


Figure 2.8 SBERT Architecture

Source: Tsang (2023)

As shown in Figure 2.8, the SBERT architecture begins by taking two input sentences, referred to as Sentence A and Sentence B. Each sentence is independently processed through a pre-trained BERT model to generate token embeddings. These embeddings capture the contextual meaning of each word in the sentence by considering the surrounding words and sentence structure.

To convert these token-level representations into a single fixed-size vector for each sentence, a pooling layer is applied. This pooling can be done in several ways, such as by averaging all token embeddings (mean pooling). The result is two sentence-level embeddings that denoted as 'u' for Sentence A and 'v' for Sentence B.

These sentence embeddings are then compared using a similarity metric, such as cosine similarity, to produce a similarity score. This score reflects how semantically similar the two sentences are.

In a practical application, Peyton and Unnikrishnan (2023) tested SBERT in a chatbot system designed to answer frequently asked question (FAQs) from prospective university

students. They compared SBERT with popular chatbot platforms such as Google Dialogflow and Microsoft QnA Maker. As illustrated in Figure 2.9, SBERT achieved the highest performance compared to all models. Hence, it shows that SBERT's superior Natural Language Understanding (NLU) capabilities, especially when dealing with varied sentence structures and paraphrased questions.

Model	Precision	Recall	F1-score
Feed Forward	0.64	0.60	0.56
Dialogflow	0.96	0.96	0.96
QnA Maker	0.96	0.96	0.95
SBERT	0.99	0.99	0.99

Figure 2.9 Model Comparison Test Results

Source: Peyton and Unnikrishnan (2023)

In conclusion, this capability is useful in developing TGS where the system must understand a user's natural language problem description like key words and match it with similar past issues or known solutions stored in a database. SBERT also has the ability to efficiently handle semantic similarity tasks, enabling faster and more accurate retrieval of relevant information.

2.6.2 Term Frequency-Inverse Document Frequency (TF-IDF)

Term Frequency–Inverse Document Frequency (TF-IDF) is used to convert text into numerical form. It highlights important words in a document by giving higher weight to words that appear frequently in a specific document but rarely in other documents. This helps the system understand which keywords are important in each text. Mao and Wang (2025) proposed an advanced search mechanism for encrypted cloud data using an improved TF-IDF weighting scheme that includes enhancements such as position weighting and word span relevance. These refinements allow the system to better score the relevance of keywords in encrypted documents. By building an inverted index and injecting virtual keywords, their method improved search accuracy while also securing the data from index inference attacks.

Although their application focused on cloud computing environments under attack, the underlying principle can be directly applied to the design of a TGS. An improved TF-IDF model can help extract key symptom descriptions from user input and match them to the most relevant troubleshooting entries, even when the queries are partially phrased or not directly aligned with predefined data.

Table 2.5: Table of Summary for Modelling

No.	Title	Author(s) / Year	Findings	Limitations
1.	Sentence-BERT: Sentence Embeddings using Siamese BERT- Networks	Reimers & Gurevych (2019)	Introduced SBERT using a Siamese BERT structure to generate efficient sentence embeddings. Enabled fast and accurate semantic similarity comparison using cosine distance.	Requires fine-tuning for specific domains to maintain high accuracy. Not ideal for word-level tasks compared to original BERT.
2.	SentenceTransformers Documentation	SentenceTransformers (2019)	Provided practical implementation and usage of SBERT for applications such as clustering, semantic search, and similarity. Supports various pretrained models.	Primarily developer-focused; lacks in-depth academic benchmarking or comparative analysis.
3.	A comparison of chatbot platforms with the state-of-the- art sentence BERT for answering online student FAQs	Peyton & Unnikrishnan (2023)	SBERT outperformed Google Dialogflow and Microsoft QnA Maker in chatbot FAQ handling with F1-score of 0.99. Effective in understanding varied query phrasings.	Focus limited to academic FAQs; performance may vary in more dynamic or broader domains.

4.	Encrypted Search Method for Cloud Computing Data Under Attack Based on TF-IDF and Apriori Algorithm	Mao & Wang (2025)	Improved TF-IDF method with position and word-span weighting enhanced semantic search accuracy (63.8%). Combined with Apriori for keyword expansion and recall improvement (81.09%).	Designed for encrypted cloud data retrieval; not directly tested in open-text NLP environments or real-time applications like TGS.
----	---	-------------------	--	--

2.7 Model Evaluation

When evaluating models that work with text or image data, cosine similarity is one of the most used methods to measure how similar two sets of data are. It calculates the cosine of the angle between two vectors, which shows how close they are in meaning or features. This method is useful because it compares the direction of the vectors rather than their size, making it a good choice when working with data from different sources or scales (Li et al., 2025). This metric ranges from -1 to 1, where a value closer to 1 implies higher similarity. It has been widely adopted in NLP for tasks involving sentence embeddings to derive fixed-size vector representations of sentences (Reimers & Gurevych, 2019).

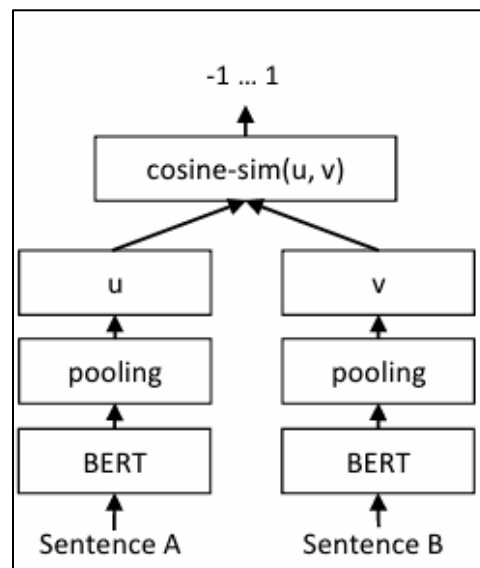


Figure 2.10 SBERT Architecture for Compute Similarity Scores

Source: Reimers and Gurevych (2019)

Cosine similarity is popular in NLP. For example, SBERT is a model that produces fixed-size sentence embeddings, which are vector representations of sentence meanings. Based on Figure 2.10, using cosine similarity can compare two sentence embeddings to see how semantically similar they are. This method is simple, fast, and often gives better results than other distance measures like Euclidean or Manhattan distances (Reimers & Gurevych, 2019).

Cosine similarity for TGS can help match user queries with the most relevant solutions. For example, when a user types in a problem, the system can convert the query into a vector and compare it with known issues in the database. By using cosine similarity, the TGS can find the closest match and suggest the correct troubleshooting steps.

Table 2.6 Table of Summary for Model Evaluation

No.	Title	Author(s)/Year	Findings	Limitations
1	A cosine similarity-based token subsampling method for vision transformer in cloud computing	Li, Q., Kaneko, H., & Meng, L. (2025)	Introduced a token subsampling technique using cosine similarity to reduce redundant tokens in Vision Transformers. Improved efficiency in cloud-based image processing without significant loss of accuracy.	Focused mainly on visual data; method not tested in natural language or multi-modal tasks.
2	Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks	Reimers, N., & Gurevych, I. (2019)	Proposed Sentence-BERT (SBERT), which creates fixed-size sentence embeddings. Improved semantic similarity tasks using cosine similarity for sentence matching.	Limited to sentence-level embedding; performance depends on quality of pre-trained language models.

2.8 Conclusion

The literature review highlights the significance role of AI tools such as NLP, OCR, and CV in enhancing the functionality of a digital TGS. Studies demonstrate the importance of preprocessing techniques in structuring raw data for effective analysis, while the integration of advanced technologies like AI-driven pipelines and real-time data processing improves operational efficiency. Case studies from various industries oversee the versatility and transformative potential of the systems, ensure their ability to reduce downtime, improve decision-making, and streamline incident management. These findings collectively validate the need for adopting innovative AI methods to develop a reliable and efficient TGS for railway operations. To conclude, this project will choose NLP for preprocessing, ETL pipeline and hybrid model which are SBERT and TF-IDF to develop the system.

CHAPTER 3

RESEARCH METHODOLOGY

3.1 Introduction

This chapter reviews the research design, business understanding, data requirements, data collection, data understanding, data preparation, development of the system and its implementation, deployment and feedback. This section outlines the methodology for the improvements of TGS in the development of the smooth system and to reach the research goals.

3.2 Research Design

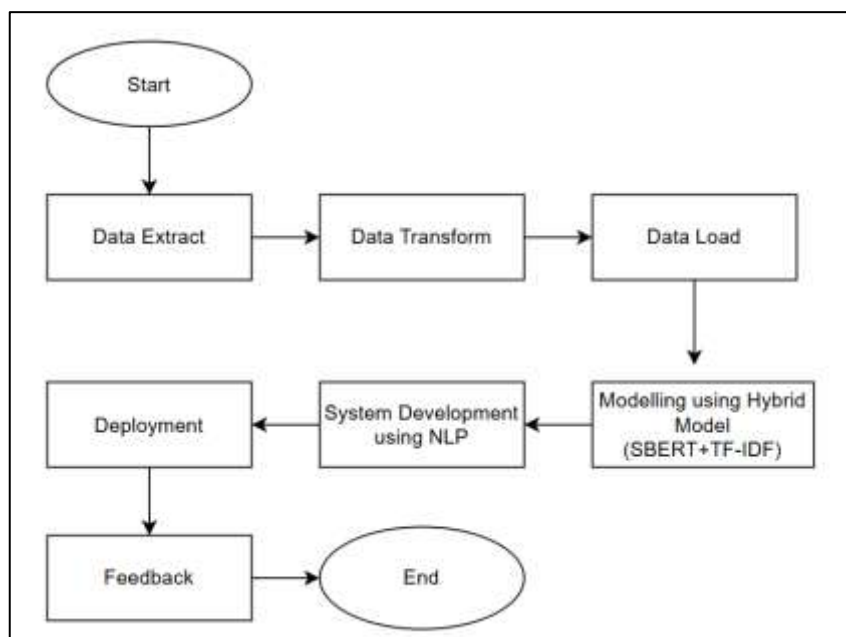


Figure 3.1 Research Framework

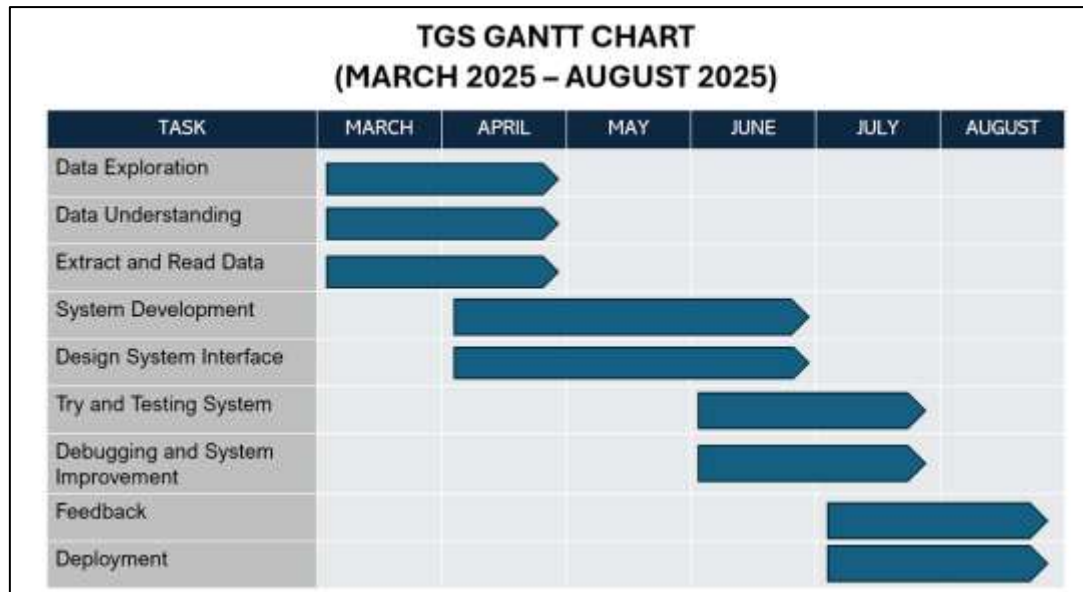


Figure 3.2 Research Gantt Chart

Based on Figure 3.1, this research follows a structured methodology to develop a TGS aimed at supporting operational failure management in Prasarana. The process begins with data extraction, where relevant data is collected from existing sources. The data is then transformed into a suitable format and loaded into the system. This step is important to ensure the quality and consistency of data used for further analysis.

After loading the data, a hybrid model combining SBERT and TF-IDF is applied for modelling. This hybrid approach allows the system to understand both the context and importance of keywords in the incident reports. The results from the model are then used in the development of the TGS using NLP techniques. NLP helps the system to give the user relevant and accurate solution recommendations based on user input and historical cases.

Once the system is developed, it goes through the deployment phase where it is implemented in the operational environment. Feedback is collected from users to assess the performance and usability of the system. This feedback is important to identify any improvements needed, and the process is repeated as necessary to enhance the system.

Figure 3.2 presents the Gantt Chart outlining development phases of the TGS. It visually represents the project timeline, highlighting key tasks such as data exploration, system development, interface design, testing, debugging, feedback collection, and final deployment. Each task is scheduled within specific months, allowing for clear tracking of progress and ensuring that all components of the system are systematically developed and integrated within the allocated timeframe.

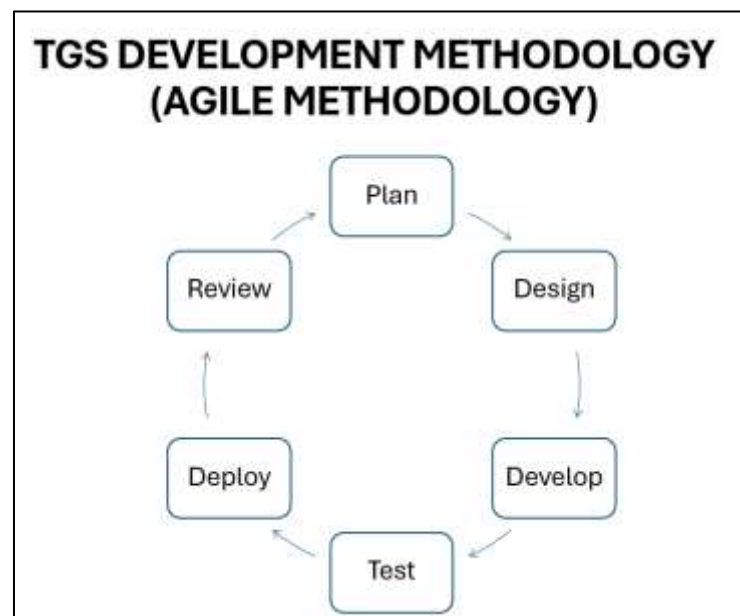


Figure 3.3 Agile Methodology

Next, Agile methodology in Figure 3.3 is used to develop TGS. Agile is suitable for this project as it allows continuous improvement through repeated development cycles. The development starts with the planning phase, where the team defines the system requirements and goals. Next, in the design phase, the interface and system structure are created. In the development phase, the actual coding and model integration are done. After development, the system is tested to ensure functionality and accuracy. The tested version is then deployed for user access. Finally, the system is reviewed based on user feedback, and the cycle begins again if further improvements are required. By combining the research flow and Agile approach, this methodology ensures that the system is built in a structured, flexible, and user-oriented manner.

3.3 Business Understanding

This research focuses on developing a digital TGS to improve how Prasarana manages railway incidents and operations. The primary question of this project is: How can a digital TGS improve operational efficiency and minimize incidents occurs in railway? Hence, identifying the root cause of operational inefficiencies and delays is crucial to ensure the system run smoothly. The main goal is the system can enhance the operation and management in railway sector and can make faster decision-making during incident or problems occurs. The objective is to develop digital TGS that making the troubleshoot faster for user especially technical and operation teams.

3.4 Analytical Approach

This research will rely on analytical approach. In this regard, the study will adopt prescriptive analytics to systematically provide solutions to these challenges in troubleshooting and operational inefficiencies that can help to improve the whole system. By integrating NLP, the system is developed to deliver real-time solution. Data pipelines ensure smooth processing and structuring of information that can deliver correctly. It will minimize downtime and errors while ensuring efficiency and safety in the railway operation of Prasarana. In this way, the system helps to make the railway operation smarter and shifts the conventional troubleshooting process to a digitalized way.

3.5 Data Requirements

To develop an effective TGS, the system requires structured and well-labeled data that accurately captures fault incidents, system responses, and alarm descriptions. Key data components include the area of concern such as equipment, system and subsystem, detailed failure scenarios, and the severity classification of each event such as major and minor. Additionally, alarm outputs from three distinct systems: the Automatic Train Supervision

(ATS), the Facility Control and Data Acquisition (FSCADA), and the Human-Machine Interface (HMI). These are essential for recognizing how faults are initially identified and reported. Furthermore, the dataset must include procedural guidance for central controllers and the local response actions taken by operational staff. These attributes collectively form the backbone for real-time diagnostics and recommendations in TGS. Lastly, the data must also be suitable for future analytics and visualization needs.

3.6 Data Collection

The data utilized in this project was primarily obtained from official documentation provided by Rapid Rail Sdn. Bhd., a subsidiary of Prasarana. The central reference for this study is the MOR document, which outlines standard operational procedures, failure response mechanisms, and technical protocols specifically for the MRT Kajang Line.

The first official version of the MOR was issued in 2020, followed by amendments in 2023 to incorporate improvements and reflect evolving operational and maintenance requirements. This document is extensively used by the Operational Control Center (OCC) and technical teams as a guideline for handling various failure scenarios in a standardized manner.

Although operational data is applied dynamically and may change based on real-time needs, the MOR serves as a stable and authoritative framework. Any updates made to operational and maintenance procedures are reflected through formal amendments to the MOR, which are subsequently integrated into systems such as the TGS developed in this project.

While preliminary work on the MOR began as early as 2018 or 2019, the most recent version in active use is the 2023 amended edition. No further changes have been recorded since. Therefore, no historical data is available, as records are not maintained or measured based on specific years but are instead revised according to operational enhancements and system updates.

3.7 Data Understanding

During this phase, data in MOR document was analysed in this stage. This document is a form of unstructured data that can use NLP to extract incident types and solutions to be solved. Hence, understanding the data will bring out operational challenges and helpful in the development of this system.

Table 3.1 Data Understanding Table

Variables	Data Types	Description
Equipment	object	Any machinery, device, or system used in operations
Failure Scenario	object	A specific situation or example that describes how a failure happen, does not work as intended or completely stops functioning.
Failure Classification	object	The category or type of failure.
ATS Alarm Description	object	This refers to the message or alert triggered by the ATS system when it detects a problem.
FSCADA Alarm Description	object	Description of the alarm or notification seen in the FSCADA system when a failure occurs.
HMI Alarm	object	Description of the alarm or notification seen alarm to appear on the HMI screen.
Description Criteria	object	A brief textual solution of the incident solution for control centre.

Local Response object

A brief textual solution proposed for the incident.

3.8 Packages and Libraries Used

```
import streamlit as st
import pandas as pd
import time
import datetime
import numpy as np
from sentence_transformers import SentenceTransformer, util
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import os
import re
import base64
```

Figure 3.4 Libraries Used

This system uses several Python packages as shown in Figure 3.4 to support the development of a digital TGS. The Streamlit package is used to create a simple and interactive web-based user interface. It allows users to upload files, enter text input, and view results directly through a web browser. Streamlit is especially useful because it does not require web programming knowledge such as HTML or JavaScript.

To manage and analyze data, the system uses Pandas, a popular data processing library in Python. Pandas provides powerful tools for reading Excel files, selecting important columns, and preparing data for further analysis. It helps make data manipulation more efficient and structured.

The time and datetime modules are used to record system response time and display current dates and times. These modules are useful for measuring the speed of generating recommendations and for adding time-based features to the system.

NumPy is a numerical computing library that supports fast operations on arrays and numerical data. In this system, NumPy helps with mathematical calculations such as similarity scoring and vector operations, which are important for comparing user input with past maintenance records.

For natural language understanding, the system uses the SentenceTransformers library. This library provides pre-trained models, such as paraphrase-MiniLM-L6-v2, that convert sentences into numerical vectors. These vectors help the system understand the meaning of user queries and match them with similar troubleshooting descriptions. The `util.pytorch_cos_sim` function from the same library calculates similarity between sentence vectors.

In addition to sentence embeddings, the system also applies TF-IDF Vectorization using the `TfidfVectorizer` from the `sklearn.feature_extraction.text` module. TF-IDF stands for Term Frequency-Inverse Document Frequency. It transforms text into numerical values that reflect how important a word is in a document. This method helps the system identify keywords that are commonly used in maintenance records.

To compare the similarity between texts, the system uses cosine similarity, a function from the `sklearn.metrics.pairwise` module. Cosine similarity measures how similar two vectors are, which helps rank the most relevant troubleshooting suggestions based on user input.

The `os` module is used for file and directory management. It helps the system read uploaded files, manage file paths, and ensure smooth operations across different operating systems.

The `re` module (regular expressions) is used for text cleaning and preprocessing. It helps remove unnecessary characters, standardize formatting, and extract useful patterns from text before analysis.

3.9 Data Preparation

Data preparation is an important step in developing TGS. This process involves collecting, cleaning, and organizing raw data to make it ready for analysis and modelling. In this study, the main data source is MOR document, which include information about equipment, failure scenarios, and response instructions. The first step in data preparation is to check the dataset to make sure important columns like "Equipment" and "Failure Scenario" exist and to remove any empty or missing data to keep the dataset clean.

After checking the data, the text is preprocessed to improve the system's ability to find the correct matches. This preprocessing converts all text to lowercase to keep it consistent. It also removes unnecessary special characters but keeps important symbols like dashes and parentheses. Extra spaces are removed to make the text uniform. Furthermore, only the first few sentences of each failure scenario are taken because these usually contain the most important information. This helps reduce irrelevant details and improves the accuracy of the system. These steps create a clear and organized dataset from the original raw data. Proper data preparation is essential because it ensures the models work with good quality data, which leads to better and more accurate troubleshooting results.

3.10 Modelling

This research employs a hybrid NLP model that combines SBERT and TF-IDF to develop a functional and intelligent TGS. The objective of this model is to automatically retrieve the most relevant historical failure scenario based on user input and to suggest appropriate corrective actions. The hybrid approach ensures the system can return accurate recommendations, thus supporting operational decision-making during real-time incidents.

$$TF - IDF(t, d) = TF(t, d) \times IDF(t) \quad (1)$$

Where:

t = Term (word)

d = Document

$$TF(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

$$IDF(t) = \log\left(\frac{N}{1+DF(t)}\right) \quad (2)$$

Where:

N = total number of documents

$DF(t)$ = Number of documents containing term t

TF-IDF is a traditional NLP technique used to quantify the importance of words in a document relative to a collection of documents. It is useful for identifying significant keywords in failure scenario records, especially in structured technical text. However, TF-IDF alone cannot understand sentence meaning, grammar, or word relationships. This limits its effectiveness in handling more complex or differently word inputs that refer to the same issue.

$$\vec{s} = \frac{1}{n} \sum_{i=1}^n \vec{h}_i \quad (3)$$

Where:

$\vec{h}_i$ = Hidden state of token i from the final layer of BERT

n = Number of tokens in the sentence

$\vec{s}$ = Sentence embedding (the final output of SBERT)

To address this limitation, SBERT is integrated into the model. SBERT is based on Bidirectional Encoder Representations from Transformers (BERT), a powerful and widely used language model developed by Google. SBERT enhances BERT by adding pooling

operations that allow it to generate fixed-size sentence embeddings. Numerical representations that preserve the full semantic meaning of sentences. This makes SBERT highly effective for tasks such as semantic similarity, textual search, question answering, and text clustering.

In this hybrid model, both TF-IDF and SBERT are used to process input alarm texts. SBERT embeddings provide the semantic context, while TF-IDF adds term-based weighting to highlight specific keywords. The input is compared to the historical dataset, and similarity scores are calculated between the new input and each record. The system then retrieves the most relevant failure scenario and displays its recommended actions, enabling a more consistent and informed response by operational staff.

Importantly, there is no training and testing phase in this model. This is because the system is designed for retrieval, not classification or prediction. The dataset serves as a static knowledge base, and the model does not learn patterns from labelled data. Instead, it compares new inputs with existing records based on similarity, making the system more interpretable and suitable for real-world deployment.

3.11 Evaluation

The evaluation in this project focuses on how well the hybrid model can find the correct failure scenario and recommendation based on what the user types into the system. To measure how similar the user's input is to the records in the dataset, the system uses cosine similarity.

$$\text{Cosine Similarity } (\vec{A}, \vec{B}) = \frac{\vec{A} \cdot \vec{B}}{||\vec{A}|| \times ||\vec{B}||} \quad (4)$$

Where:

$A \cdot B$ = Dot product of vectors A and B

$||A||$ and $||B||$ = Magnitudes (lengths) of the vectors

Cosine similarity is a method commonly used in text analysis. It calculates how similar two sentences are by comparing their vector representations. These vectors are created using SBERT, which turns each sentence into a set of numbers that represent its meaning. Cosine similarity gives a score between 0 and 1. A score closer to 1 means the two sentences have similar meanings.

When a user enters a failure description, the system compares it to all failure records in the dataset using cosine similarity. The system then selects the most similar record and shows the related recommendation, which includes the Chief Controller's instruction and the Local Response. This helps staff find the most relevant action based on past incidents.

3.12 ETL Pipeline

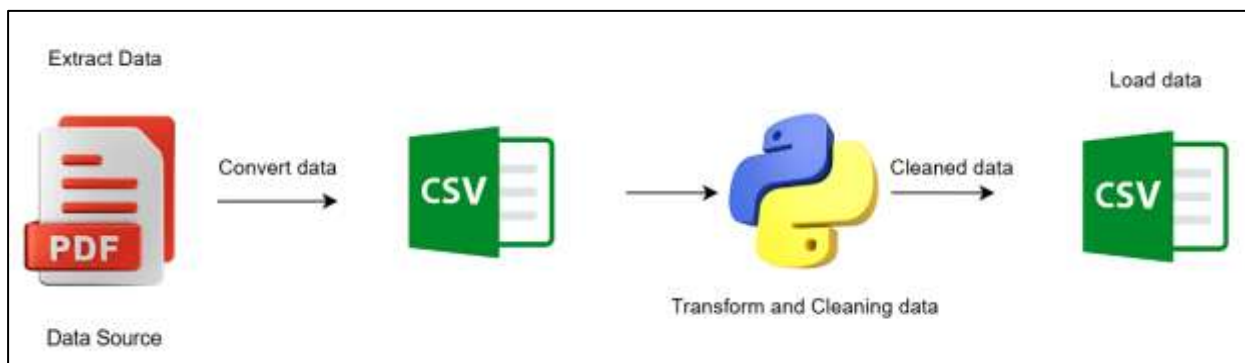


Figure 3.5 ETL Pipeline Structure

The ETL process in this study as illustrates in Figure 3.5 involves three main steps: Extract, Transform, and Load. These steps were applied to the MOR data to prepare it for further analysis.

In the Extract phase, the original MOR data was provided in a PDF file format. Since PDF is not suitable for direct data processing in most programming tools, the information had

to be manually extracted. This was done by copying and pasting the content from the PDF file into a CSV file. Although this step was done manually, it was necessary to make the data usable for the next phases of processing.

During the Transform phase, the data from the CSV file was cleaned and prepared using Python. Several data cleaning techniques were applied, including handling missing values, converting text to lowercase, removing special characters, and trimming unnecessary spaces. In some cases, only the most important parts of the text such as the first few sentences of a description were kept making the data more relevant. These steps helped ensure that the data was consistent, accurate, and ready for analysis. All the transformation phases are done in Python.

Finally, in the Load phase, the cleaned data was saved into a new CSV file. This final version of the data can now be used for further processing, analysis, or model development. Storing the cleaned data in a separate file ensures that the original raw data remains untouched, and that the cleaned version is ready for use in future tasks.

3.13 Deployment

Since the system was developed using Streamlit, it is deployed using the same framework. Streamlit provides an easy and efficient way to turn Python scripts into interactive web applications. To deploy the system, the project files must first be uploaded to a GitHub repository. This includes the main Python file, a requirements file listing all necessary packages, and any related data files. After the files are properly organized and uploaded, the user needs to sign in to the Streamlit Community Cloud and connect their GitHub account. Once connected, the user can select the desired repository and choose the main Python file to launch the application. Streamlit will automatically install the required packages and run the app. After successful deployment, a link to the live application will be generated. This method is user-friendly and suitable for academic or personal use, especially for those who wish to share their project online without needing advanced technical setup.

3.14 Feedback

The feedback phase helps to improve digital TGS after system can be deployed. Continuous surveys from the staff will be one of major thing to monitor the system. This phase will ensure to see which part need to be improved and the system is safe and works accurately for continuous improvements.

3.15 Conclusion

In this chapter, the data and methodologies that were employed to develop the TGS are discussed in detail. Therefore, both the processes for the collection of data, its preparation, and how the design of the ETL pipeline was carried out to achieve the object of the project is outlined. The results and implementation of the system will be elaborated further in Chapter 4.

CHAPTER 4

DATA ANALYSIS, RESULTS AND DISCUSSION

4.1 Introduction

This chapter will focus on the outlined methodology in the previous chapter and will show the results of the analysis based on the objective of this study stated in Chapter 1. This chapter will discuss the data understanding, data cleaning, data preparation, exploratory data analysis, and deployment phase of the model

4.2 Data Collection

```
[ ] import pandas as pd

# Load the uploaded file
file_path = "/content/MOR_MGL.csv"
df = pd.read_csv(file_path)

# Show head of the dataset
df.head()
```

	Equipment	Failure Scenario	Failure Classification	ATS Alarm Description	PSCADA Alarm Description	HMI Alarm	Guidelines for the Chief Controller	Local Response
0	TMS	Failure of 1 TMS Central Control Unit (CCU)	Minor	Train Control Minor Fault	NaN	Train Control Minor Fault	Train Entering service. Not allowed to enter s...	Reset CCU
1	TMS	Failure of 2 TMS Central Control Unit (CCU)	Major	Train Control Minor Fault	NaN	Train Control Major Fault	Train Entering service. Not allowed to enter s...	Reset CCU. If not success, train need to be co...
2	TMS	Failure of Single Multifunction Vehicle Bus (M...	Minor	Train control equipment minor fault	NaN	Train control equipment minor fault	Train Entering service. Not allowed to enter s...	Check on HMI and identify which equipment caus...
3	TMS	Failure of Double Multifunction Vehicle Bus (M...	Major	Train control equipment minor fault	NaN	Train control equipment major fault	Train Entering service. Not allowed to enter s...	Check on HMI and identify which equipment caus...
4	Power	Loss of DC Power by Train	Major	1st alarm appear Loss of traction energy and 3...	NaN	Loss of Traction Energy	Train Entering service. Not allowed to enter s...	Qualified train driver verify the alarm, train ...

Figure 4.1 Raw Data from Minimum Operation Requirement (MOR)

Based on Figure 4.1, the data used in this project was collected primarily from official technical documentation provided by Rapid Rail Sdn Bhd. MOR outlines various equipment failure scenarios, alarm descriptions, failure classifications, and corresponding operational responses in MRT Kajang Line (KGL). This document was supplied in PDF format and

contains structured information related to systems such as ATS, TMS, FSCADA, and HMI. Key data extracted includes equipment types, possible fault conditions, associated alarm messages: ATS, FSCADA, and HMI, classification of failure levels: Minor and Major, decision criteria for the Chief Controller and expected field or local response actions. To ensure accuracy, the extracted data was manually reviewed and cross-verified with the original document. The information was then organized into structured tables using Pandas and stored in CSV format.

4.3 Data Understanding and Exploration

```
[ ] duplicates = df.duplicated().sum()
print("\nDuplications:", duplicates)
```

⇒ Duplications: 0

Figure 4.2 Check Duplicate Data

Based on Figure 4.2, the dataset does not contain any duplicate entries, ensuring that each failure scenario and its corresponding operational response are unique and reliable for analysis and system implementation.

```
# Show basic info of the dataset
df.info()
```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 93 entries, 0 to 92
Data columns (total 8 columns):
Column Non-Null Count Dtype

0 Equipment 93 non-null object
1 Failure Scenario 93 non-null object
2 Failure Classification 93 non-null object
3 ATS Alarm Description 50 non-null object
4 FSCADA Alarm Description 7 non-null object
5 HMI Alarm 62 non-null object
6 Guidelines for the Chief Controller 93 non-null object
7 Local Response 68 non-null object
dtypes: object(8)
memory usage: 5.8+ KB

```
[ ] # Check for missing values and duplicates
missing_text = df.isna().sum()
print("Missing Text:")
print(missing_text)
```

⇒ Missing Text:
Equipment 0
Failure Scenario 0
Failure Classification 0
ATS Alarm Description 41
FSCADA Alarm Description 84
HMI Alarm 29
Guidelines for the Chief Controller 0
Local Response 23
dtype: int64

Figure 4.3 Check Data Type and Missing Text

Figure 4.3 presents the basic structure and missing value summary of the dataset. The dataset contains 91 entries and 8 columns, all of which are of object data type. The columns 'Equipment', 'Failure Scenario', 'Failure Classification', and 'Guidelines for the Chief Controller' are complete, with no missing values. However, several other columns contain missing values. The 'ATS Alarm Description' has 41 missing entries, 'FSCADA Alarm Description' has the highest number of missing values at 84, and 'HMI Alarm' has 29 missing values. Additionally, 'Local Response' is missing in 23 records. These missing values indicate that in MOR, not all alarm descriptions or solutions are involved or required. Therefore, we must fill in the missing values with suitable text to avoid misrepresentation in the analysis and development of the TGS.

```

[ ] # Fill actual NaN values appropriately
df_cleaned['HMI Alarm'] = df_cleaned['HMI Alarm'].fillna('no alarm triggered')
df_cleaned['FSCADA Alarm Description'] = df_cleaned['FSCADA Alarm Description'].fillna('no alarm triggered')
df_cleaned['ATS Alarm Description'] = df_cleaned['ATS Alarm Description'].fillna('no alarm triggered')

df_cleaned['Local Response'] = df_cleaned['Local Response'].fillna('no solutions provided')
df_cleaned['Guidelines for the Chief Controller'] = df_cleaned['Guidelines for the Chief Controller'].fillna('no solutions provided')
df_cleaned.head()

```

	Equipment	Failure Scenario	Failure Classification	ATS Alarm Description	FSCADA Alarm Description	HMI Alarm	Guidelines for the Chief Controller	Local Response
0	TMS	Failure of 1 TMS Central Control Unit CCU	Minor	Train Control Minor Fault	No alarm triggered	Train Control Minor Fault	Train Entering service: Not allowed to enter s...	Reset CCU
1	TMS	Failure of 2 TMS Central Control Unit CCU	Major	Train Control Minor Fault	No alarm triggered	Train Control Major Fault	Train Entering service: Not allowed to enter s...	Reset CCU. If not success, train need to be co...
2	TMS	Failure of Single Multifunction Vehicle Bus MV...	Minor	Train control equipment minor fault	No alarm triggered	Train control equipment minor fault	Train Entering service: Not allowed to enter s...	Check on HMI and identify which equipment caus...
3	TMS	Failure of Double Multifunction Vehicle Bus MV...	Major	Train control equipment minor fault	No alarm triggered	Train control equipment major fault	Train Entering service: Not allowed to enter s...	Check on HMI and identify which equipment caus...
4	Power	Loss of DC Power by Train	Major	1st alarm appear Loss of traction energy and 3...	No alarm triggered	Loss of Traction Energy	Train Entering service: Not allowed to enter s...	Qualified train driver verify the alarm, train...

Figure 4.4 Fill in Missing Text

Based on Figure 4.4, to ensure consistency and clarity in the dataset, missing values were handled appropriately based on the context of each field. As shown in the output, missing values in the 'HMI Alarm', 'FSCADA Alarm Description', and 'ATS Alarm Description' columns were filled with the phrase 'No alarm triggered'. This replacement was made because these alarms were intentionally left blank in the original data to indicate that no alarm was activated for that specific failure scenario. Similarly, missing values in 'Local Response' and 'Guidelines for the Chief Controller' were filled with 'No solutions provided', as their absence suggests that no action or guideline was necessary for those cases. This method preserves the

meaning intended in the original dataset and avoids misinterpretation that could occur if the missing values were left untreated or incorrectly imputed.

4.4 Data Preparation

The data preparation phase involved transforming the raw extracted information into a clean and structured format suitable for use in the TGS development. Throughout this section, the comprehensive data preparation process will enable us to derive meaningful insight from the dataset.

4.4.1 Data Cleaning

```
def preprocess_text_for_matching(text):
    """Enhanced text preprocessing for better matching"""
    import re
    if not text or pd.isna(text):
        return ""

    # Convert to lowercase and remove extra whitespace
    text = str(text).lower().strip()

    # Remove special characters but keep important ones
    text = re.sub(r'["\w\s\-\_\\\/]', ' ', text)

    # Replace multiple spaces with single space
    text = re.sub(r'\s+', ' ', text)

    return text

def extract_first_sentences(text, num_sentences=3):
    """Extract first N sentences for better matching on key information"""
    if not text or pd.isna(text):
        return ""

    import re
    # Split by sentence endings
    sentences = re.split(r'[.!?]+', str(text))
    # Take first N sentences and clean them
    first_sentences = []
    for i, sentence in enumerate(sentences[:num_sentences]):
        sentence = sentence.strip()
        if sentence: # Only add non-empty sentences
            first_sentences.append(sentence)

    return '. '.join(first_sentences) if first_sentences else str(text)
```

Figure 4.5 Data Cleaning Process

In this project, data cleaning is performed in Figure 4.5 as part of the Natural Language Processing (NLP) preprocessing step. The main goal is to ensure that the input text is clean, consistent, and suitable for similarity comparison using SBERT models. These models, such as *all-MiniLM-L6-v2* and *paraphrase-MPNet*, are transformer-based and able to understand context. Since they are pretrained on natural language, they already recognize grammar, sentence structure, synonyms, and different word forms. As a result, advanced preprocessing techniques like lemmatization, stemming, stopword removal, or tokenization are not necessary and it can reduce the model's performance. Instead, only basic cleaning is applied, such as converting text to lowercase if it is needed, removing unnecessary HTML tags, normalizing

spaces, and removing non-informative characters. Important symbols such as hyphens, slashes, and brackets are kept because they may carry technical meaning.

The `preprocess_text_for_matching` function carries out these cleaning steps. First, it checks if the text is missing or empty. If the text is valid, it converts it to lowercase, removes extra spaces, and uses regular expressions to filter out unwanted characters. It also replaces multiple spaces with a single space to keep the format consistent. This helps reduce noise and ensures that the text is in a standard format, making comparison more reliable.

To improve the focus of the input text, the `extract_first_sentences` function is used to select only the first few sentences which is usually three. This is based on the idea that the most important information in technical texts is often stated at the beginning. The function splits the text using punctuation that ends a sentence, cleans the selected parts, and combines them into a shorter text for analysis. This helps the system work more efficiently and increases the accuracy of the similarity-matching engine by focusing on the most useful part of the text.

4.5 Exploratory Data Analysis (EDA)

In this phase, Python was used as the primary tool to explore and understand the underlying patterns within the dataset. Various libraries such as Pandas, Matplotlib, Seaborn, and were employed to perform both descriptive and visual analysis.



Figure 4.6 Frequency of Failure Classification

The bar chart shows in Figure 4.6 the frequency of failure classifications in the dataset, specifically between minor and major failures. Minor failures have a count of approximately 43. Major failures have a slightly higher count, at around 47. This indicates that major failures occur slightly more frequently than minor ones in the collected data. The relatively balanced distribution suggests that both categories are significant in railway operations and must be equally considered in the design of the TGS. Additionally, this frequency may influence how response actions are prioritized, with major failures requiring more urgent attention or more detailed handling procedures.



Figure 4.7 Word Cloud for HMI, FSCADA and ATS Alarm Description

Word cloud in Figure 4.7 was used to analyse the extracted text data from the Minimum Operating Requirement (MOR) document to identify the terms and keywords that were mentioned most frequently. This method helps provide a visual summary of the most dominant alarm status within the dataset. The size of each word in the cloud corresponds to its frequency in the text, meaning that words appearing more often are displayed in larger font sizes. Through this analysis, frequently occurring terms such as “Major,” “triggered,” and “fault” were easily identified. This not only confirms the relevance of those terms in the troubleshooting process but also helps highlight the core focus areas in railway fault management. Additionally, word cloud analysis is useful in the early stages of data exploration to guide the development of rule-based systems or natural language processing components, as it provides insight into the structure and density of the source content.

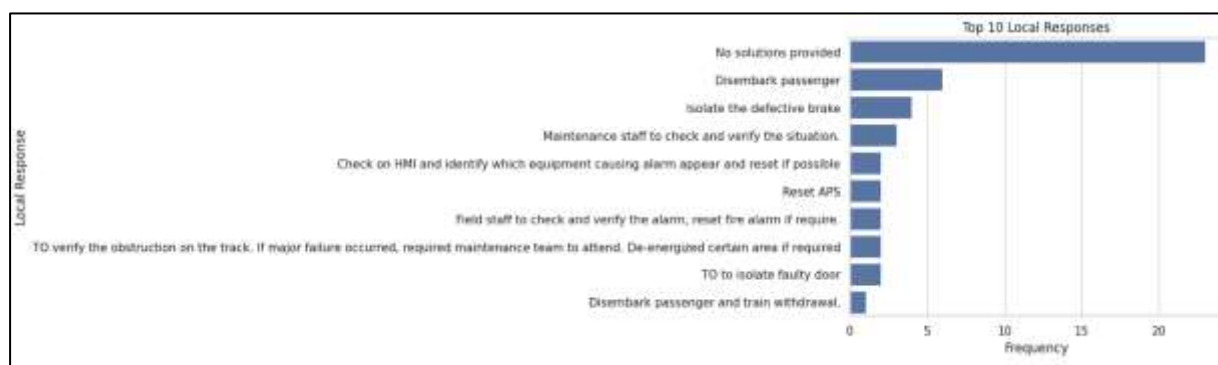


Figure 4.8 Top 10 of Guidelines for the Chief Controller

Figure 4.8 illustrates the Top 10 Guidelines for the Chief Controller, presenting the most frequently selected guidelines sentence. The two most preferred guidelines, with a frequency of approximately 7.5 each are: "Train entering service: Not allowed to enter service; train already in service: Exit at terminus station" and "Train entering service: Not allowed to enter service; train already in service: Immediate withdrawal from service." These results indicate a strong emphasis on operational safety by restricting new entries and ensuring the proper removal of trains already in service. Other highly rated guidelines include "Train entering service: Not allowed to enter service; train already in service: Removal at fleet

reduction" with approximately 5.5 and "Exit at terminus station" with approximately 5.0. Interestingly, "Train entering service: Allowed to enter service; train already in service: Remain in service" also shows moderate value, which is around 4.5, suggesting specific conditions where continued operation is acceptable. The remaining guidelines, with lower frequencies, involve measures such as remote intervention or conditional withdrawal. Overall, the data suggests a strong prioritization of managing train movement to maintain safety and efficiency.

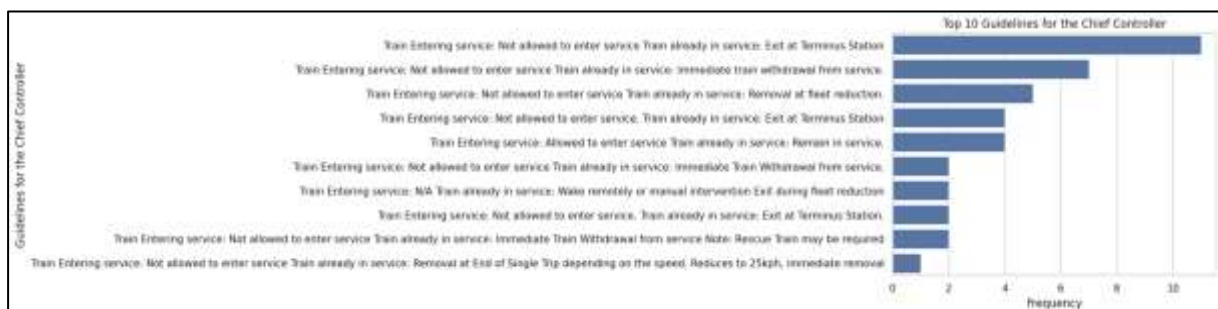


Figure 4.9 Top 10 of Local Response

Figure 4.9 shows the Top 10 Local Responses to operational incidents. The most frequent response, with a significantly higher frequency with approximately over 20 than the rest, is "No solutions provided," indicating a critical gap in documented or prepared responses. This suggests that in many cases, staff encounter situations without clear procedures to follow. Other common responses include "Disembark passenger" and "Isolate the defective brake," both with frequencies around 5. Additionally, responses such as "Maintenance staff to check and verify the situation," "Check on HMI and identify the equipment causing the alarm and reset if possible," and "Reset APS" were moderately frequent. Further responses include verification and resetting actions by field staff or train operators (TO), such as isolating faulty components or managing track obstructions. The least reported response, "Disembark passenger and train withdrawal," appears with a frequency of only about 0.5. Overall, the chart emphasizes a pattern of immediate, practical actions in the absence of formalized solutions, highlighting a need for improved incident response planning.

4.6 Hybrid Model Development

```
# --- Hybrid Model Setup (SBERT + TF-IDF + Cosine Similarity) ---
@st.cache_resource
def load_models():
    """Load both SBERT model and initialize TF-IDF vectorizer"""
    try:
        sbert_model = SentenceTransformer('paraphrase-MiniLM-L6-v2')
        tfidf_vectorizer = TfidfVectorizer(
            max_features=1000,
            stop_words='english',
            ngram_range=(1, 2),
            lowercase=True
        )
        return sbert_model, tfidf_vectorizer
    except Exception as e:
        st.error(f"Error loading models: {str(e)}")
        st.error("Please ensure required packages are installed: pip install sentence-transformers scikit-learn")
        return None, None

sbert_model, tfidf_vectorizer = load_models()
SBERT_THRESHOLD = 0.4
TFIDF_THRESHOLD = 0.15
HYBRID_WEIGHT_SBERT = 0.6
HYBRID_WEIGHT_TFIDF = 0.4
```

Figure 4.10 Develop Hybrid Model

The function `load_models()` in Figure 4.10 was created to load two important components used for matching user input with troubleshooting records: the SBERT model and the TF-IDF vectorizer. SBERT (Sentence-BERT) is a semantic model that captures the meaning of entire sentences, while TF-IDF (Term Frequency-Inverse Document Frequency) is a rule-based approach that focuses on the frequency of specific keywords in the text. By combining both approaches, the system can deliver more accurate and meaningful suggestions.

Inside the function, the SBERT model 'paraphrase-MiniLM-L6-v2' is loaded using the `SentenceTransformer` class. This model is known for its ability to efficiently compute sentence similarity. At the same time, the TF-IDF vectorizer is initialized with settings such as limiting to the top 1000 most relevant terms, removing common English stopwords, allowing both single words (unigrams) and two-word combinations (bigrams), and converting all text to lowercase for consistency. These preprocessing steps help improve matching accuracy by focusing on the most important information.

If both models are loaded successfully, the function returns them. However, if there's an error such as missing required packages, the function will display an error message using Streamlit and return `None` for both models. This prevents the app from crashing and helps the user identify what needs to be fixed.

After defining the function, the models are loaded and assigned to `sbert_model` and `tfidf_vectorizer`. Next, similarity thresholds and weight values are defined to guide the hybrid matching process. For SBERT, a similarity threshold of 0.4 is set. This means that only results with a similarity score of 0.4 or higher will be considered a good semantic match. The score ranges from 0 to 1, where 1 means an exact or very close match in meaning. A lower threshold like 0.4 helps the system be more sensitive and inclusive of relevant results, even if they are not perfect matches.

Similarly, for TF-IDF, the threshold is set at 0.15, meaning keyword matches with a score above 0.15 are accepted. This score reflects how important a keyword is to a specific document compared to all documents. A lower threshold ensures that more potentially useful keyword matches are included in the recommendations.

Finally, hybrid weights are applied to combine the scores from both models: 60% weight for SBERT and 40% for TF-IDF. This weighting gives more importance to sentence meaning while still considering keyword relevance. Overall, the use of thresholds and weights helps filter out weak matches and ensures that users receive useful and relevant troubleshooting suggestions during railway failure incidents.

4.7 Hybrid Model Evaluation

```
# SBERT similarity on processed texts
all_texts_sbert = processed_list + [processed_input]
sbert_embeddings = sbert_model.encode(all_texts_sbert)
sbert_similarities = util.cos_sim(sbert_embeddings[-1], sbert_embeddings[:-1])

# TF-IDF similarity on processed texts
tfidf_matrix = tfidf_vectorizer.fit_transform(all_texts_sbert)
tfidf_similarities = cosine_similarity(tfidf_matrix[-1], tfidf_matrix[:-1])

# Calculate individual and hybrid scores
results = []
for i in range(len(predefined_list)):
    sbert_score = sbert_similarities[0][i].item()
    tfidf_score = tfidf_similarities[0][i]
    hybrid_score = (HYBRID_WEIGHT_SBERT * sbert_score +
                    HYBRID_WEIGHT_TFIDF * tfidf_score)
```

Figure 4.11 Evaluation using Cosine Similarity

For evaluation, the code in Figure 4.11 is not meant to evaluate the performance of the overall model like in traditional machine learning evaluation such as using a error metrices or F1-score. Instead, it performs a real-time similarity check whenever a user enters a query into the TGS. The goal is to find the most relevant solution that matches the user's input in failure scenario search.

First, the code calculates SBERT similarity by combining all preprocessed historical texts (processed_list) with the new user input (processed_input) into a single list called all_texts_sbert. These texts are then converted into vector embeddings using the SBERT model with sbert_model.encode(). The similarity between the user's input (last vector) and each historical record (all previous vectors) is calculated using util.cos_sim(), which computes cosine similarity. This returns a score between -1 and 1, where 1 indicates high similarity.

Next, the same list of texts is used to compute TF-IDF similarity. The tfidf_vectorizer.fit_transform() function turns the text into TF-IDF vectors. Then, cosine_similarity() is used again to compare the TF-IDF vector of the user's input with the vectors of the other records. This also results in a score between 0 and 1, where higher values mean more similarity in terms of shared keywords.

Overall, both methods use Cosine Similarity to quantify how closely the new input matches past records that focusing on sentence SBERT and TF-IDF.

4.8 Graphical User Interface (GUI)

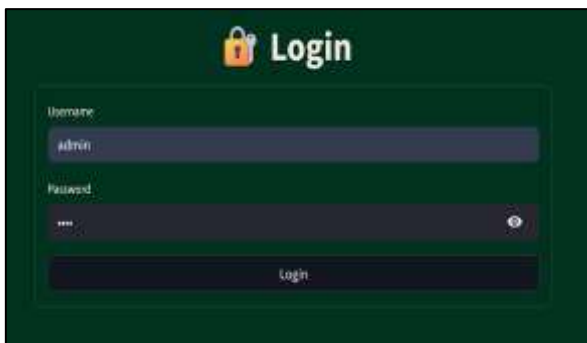


Figure 4.12 User Authentication

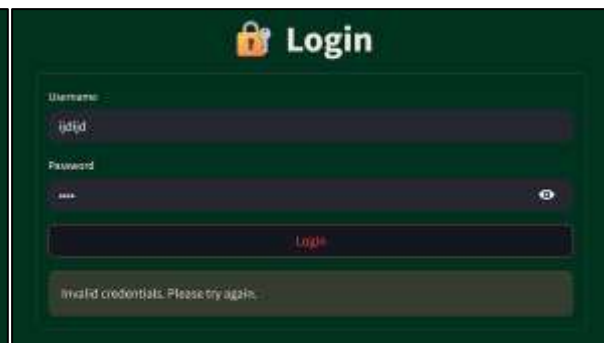


Figure 4.13 Access Denied to Login

Figure 4.12 and Figure 4.13 display the user authentication interface, where authorized personnel are required to log in before accessing the TGS. This ensures that only registered users can interact with the system, maintaining data privacy and system security.

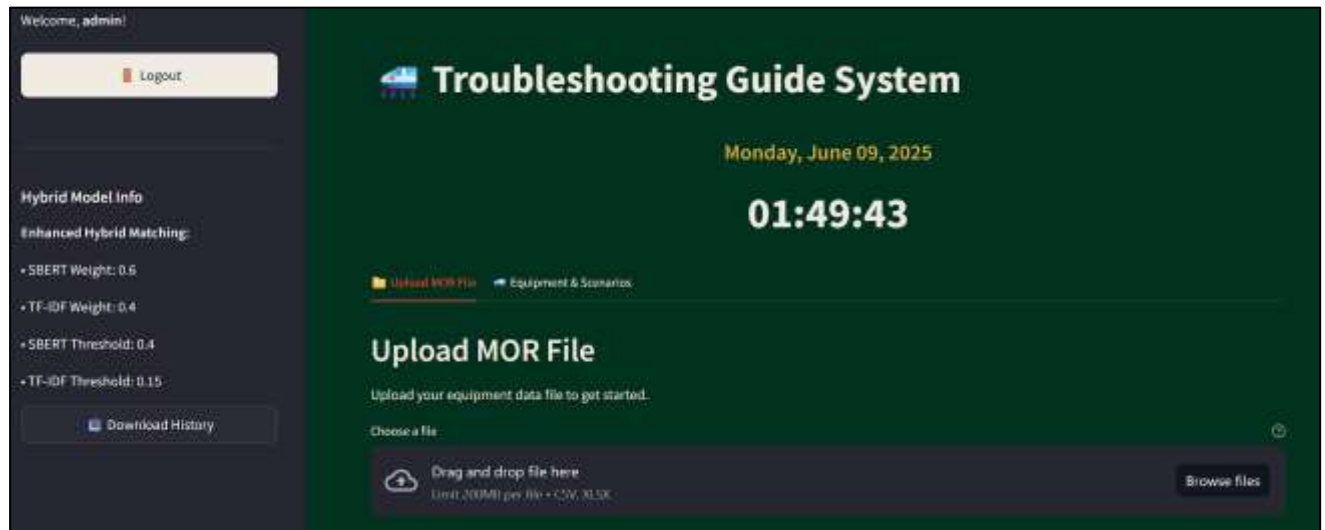


Figure 4.14 User Interface and Upload Data

Figure 4.14 illustrates the feature that allows users to upload relevant maintenance or incident datasets into the system. Users can upload files in either CSV or XLSX format, providing flexibility in data sources. Once uploaded, the data will undergo preprocessing and be stored for further analysis and matching with user queries. Additionally, a log-out button is available at the top left corner of the interface for secure session management. The interface also displays model threshold scores, which inform users about the predefined similarity thresholds used for generating recommendations.

MOR_KGL.csv 28.2KB

File "MOR_KGL.csv" uploaded successfully!

Data Preview

	Equipment	Failure Scenario	Failure Classification	ATS Alarm Description
0	TMS	Failure of 1 TMS Central Control Unit (CCU)	Minor	Train Control Minor Fault
1	TMS	Failure of 2 TMS Central Control Unit (CCU)	Major	Train Control Minor Fault
2	TMS	Failure of Single Multifunction Vehicle Bus (MVB) Network	Minor	Train control equipment minor fault
3	TMS	Failure of Double Multifunction Vehicle Bus (MVB) Network	Major	Train control equipment minor fault
4	Power	Loss of DC Power by Train	Major	1st alarm appear Loss of traction energy and 3AC power supply fault

Total Records

91

Equipment Types

34

Columns

8

Figure 4.15 Data Preview

Figure 4.15 presents a preview of the uploaded dataset, enabling users to verify the data content before proceeding. It provides transparency and allows users to ensure that the correct file has been uploaded and is ready for processing.

Troubleshooting Guide System

Monday, June 09, 2025

01:51:28

Upload MOR File

Equipment & Scenario

Select Area of Concern

TMS

Power

Traction

Brake

Suspension System

Train Doors

Auxiliary Driving Console

Train Coupler

Axe

Lighting

Pneumatic System

Air Conditioning Unit

Figure 4.16 Selection Area of Concern

Figure 4.16 presents the interface that allows users to select the specific area or component of concern: equipment, subsystem, symptoms, and signalling. These options are extracted directly from the uploaded dataset and displayed for user selection. By choosing a particular category, the system filters the records and enable more focused analysis and generating recommendations that are relevant to the selected area.



Figure 4.17 User can Search and Select Failure Scenario

Figure 4.17 demonstrates the functionality where users can either input specific keywords or select from predefined failure scenarios to initiate a search. The input serves as a query for the system to identify and retrieve similar past incidents from the dataset. The similarity search is conducted using a hybrid model that combines TF-IDF and SBERT. This combination enables the system to assess both the lexical similarity and the semantic meaning of the input query, thereby providing more accurate and relevant matches.



Figure 4.18 Model Comparison and Finding Best Match Failure Scenario

Figure 4.18 shows how the system compares two different ways of finding similar past failure cases: one method focuses on matching keywords (called TF-IDF), and the other understands the meaning of the text (called SBERT). The system uses a method called cosine similarity to measure how close the user's input is to past records. After comparing both methods, the system shows the result with the highest matching score. This helps users quickly find the most relevant past problem and its solution

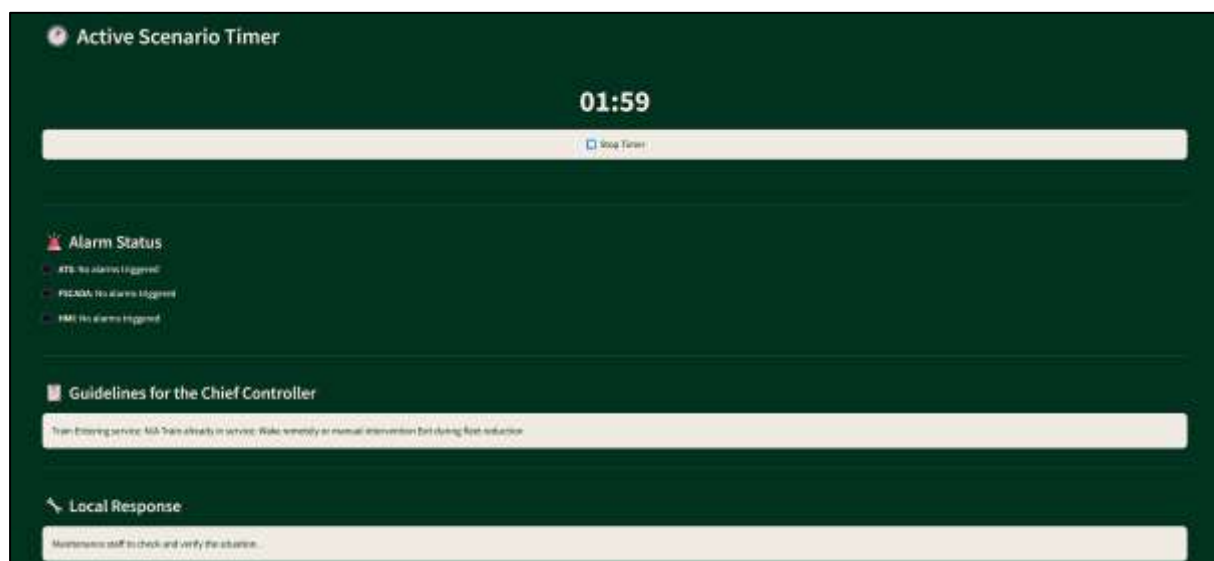


Figure 4.19 Timer, Alarm Status Triggered and Solutions Recommendations

Figure 4.19 shows the final output interface, where the matched scenario results in an alarm being triggered and a timer being activated. The system then displays the recommended solutions for the identified issue, helping users respond promptly and efficiently.

	A	B	C	D	E	F	G	H	I	J
1	Staff ID	Start Time	Stop Time	Date	Equipment	Failure	St. Status	Guideline	Local Resg	Duration (mins)
2	admin	27/5/2025 17:27	27/5/2025 17:28	27/5/2025	Train Door	Train Door	Resolved	Train Enter	Cycle door	0
3	admin	27/5/2025 22:16	27/5/2025 22:16	27/5/2025	Train Door	Failure/L	Resolved	Train Enter	i. Quality tr	0
4	admin	27/5/2025 22:35	27/5/2025 22:59	27/5/2025	Lightning	Failure of 1	Failed	Train Entering	service:	23
5	admin	27/5/2025 23:00	27/5/2025 23:01	27/5/2025	Power	Failure of 1	Resolved	Train Entering	service:	1
6	admin	27/5/2025 23:04	27/5/2025 23:07	27/5/2025	Train Door	All door fai	Resolved	Train Enter	Disembark	3
7	admin	28/5/2025 11:35	28/5/2025 11:35	28/5/2025	Train Coup	Automatic	Resolved	Train Enter	Field staff	0
8	admin	28/5/2025 22:56	28/5/2025 23:10	28/5/2025	Radio	Failure of 1	Failed	Train Entering	service:	13
9	admin	29/5/2025 9:22	29/5/2025 9:22	29/5/2025	Train Door	All door fai	Resolved	Train Enter	Disembark	0
10	admin	29/5/2025 16:00	29/5/2025 16:01	29/5/2025	TMS	Failure of 5	Resolved	Train Enter	Check on	0
11	admin	29/5/2025 16:03	29/5/2025 16:04	29/5/2025	Axle	Locked Axle	Resolved	Train Enter	Disembark	1
12	admin	30/5/2025 9:19	30/5/2025 9:20	30/5/2025	VATC	Single VAT	Resolved	Train Enter	1st attempt	1
13	admin	30/5/2025 11:04	30/5/2025 11:20	30/5/2025	Excessive	1 If noise is	Failed	Train	Note: i. Mo	15
14	admin	30/5/2025 11:34	30/5/2025 11:37	30/5/2025	Wayside	Radio netw	Resolved	i. Reset co	Hard Rese	2
15	admin	2/6/2025 15:59	2/6/2025 16:00	2/6/2025	Brake	Failure of 5	Resolved	Train Enter	Disembark	0
16	admin	2/6/2025 23:41	2/6/2025 23:42	2/6/2025	750 VDC	Breaker fai	Resolved	i. Bypass d	Maintenan	0
17	admin	4/6/2025 21:02	4/6/2025 21:02	4/6/2025	Train Door	Train door	Resolved	Train Enter	Disembark	0
18	admin	4/6/2025 21:33	4/6/2025 21:44	4/6/2025	Train Door	Train door	Failed	Train Enter	Disembark	10
19	admin	5/6/2025 11:43	5/6/2025 11:50	5/6/2025	Brake	Failure of 5	Failed	Train Enter	Disembark	7
20	admin	5/6/2025 11:52	5/6/2025 11:54	5/6/2025	AT5	Centralize	Resolved	i. Reboot A	i. Maintenu	2
21	admin	5/6/2025 12:22	5/6/2025 12:23	5/6/2025	Train Door	Train door	Resolved	Train Enter	Disembark	1
22	admin	5/6/2025 20:28	5/6/2025 20:33	5/6/2025	Axle	Locked Axle	Resolved	Train Enter	Disembark	4
23	admin	5/6/2025 23:24	5/6/2025 23:24	5/6/2025	Brake	Brake Yell	Resolved	Train Entering	service:	0

Figure 4.20 Data Logged in a CSV

The dataset captured upon the activation of the Download History button in Figure 4.20 represents a detailed event log of equipment-related failures and resolutions in railway operations. The data includes fields such as staff ID, start and stop time, failure date, equipment involved, failure description and status, applied guideline, location of response, and duration in minutes.

CHAPTER 5

CONCLUSION AND RECOMMENDATIONS

5.1 Introduction

This chapter presents the expected outcomes of the research and the potential impact of the digital TGS on railway operations. It outlines the anticipated improvements in operational efficiency, incident management, and decision-making processes. Additionally, this chapter concludes the study by summarizing the key findings and contributions, highlighting how the proposed system addresses the challenges faced by Prasarana's railway network and sets the stage for future enhancements.

5.2 Conclusion

This project has successfully developed an intelligent and user-friendly TGS to assist users in identifying appropriate solutions based on textual inputs. By combining methods such as TF-IDF with SBERT, the system effectively matches user queries with relevant troubleshooting actions. This hybrid approach has proven to enhance the accuracy and relevance of the recommendation provided

The project began by converting data from PDF format into a structured CSV dataset. These records were then cleaned and prepared using a systematic data preprocessing pipeline developed in Python. The process included standardizing text, removing unnecessary characters, and extracting the most meaningful content. This ensured that the quality and consistency of the data used in the system.

A web-based interface was created using Streamlit, allowing users to interact with the system in real time. This interface makes the tool accessible and practical for the staff to access. Overall, the system contributes to improved maintenance decision-making and supports efforts to increase operational reliability and efficiency. It aligns well with Prasarana's goal of f continuous improvement and reflects its cultural belief: own it, being proactive, act responsibly, and speak up and listen.

One main limitation of this study is the system's reliance on the quality and completeness of the fixed troubleshooting data. If certain issues or rare cases are not well covered in the existing solution records, the system may fail to give accurate or helpful recommendations. Although the combination of TF-IDF and SBERT improves matching performance, it may still have difficulty understanding complex or unclear user queries. The system's ability to respond to different types of input depends on how varied and detailed the fixed data is. In addition, due to time and resource limits, the system was not tested widely across multiple departments or with many users, which may affect how well the results apply in different work environments or organizations.

5.3 Recommendation

Several recommendations are proposed for further enhancement to strengthen the effectiveness of the developed system. Firstly, the implementation of automated data extraction from PDF files to CSV format using Optical Character Recognition (OCR). This enhancement would significantly reduce manual workload, lower the risk of human error, and improve the efficiency and scalability of ETL pipeline.

Furthermore, the integration of automated dashboarding capabilities is recommended to facilitate the visualization of failure trends and provide actionable insights to users. This feature would enable operational staff and decision-makers to monitor recurring faults, track key performance indicators, and assess maintenance patterns through interactive visual analytics, thereby supporting data-driven decision-making process.

Lastly, it is advisable to implement a user feedback mechanism within the system interface. Allowing users to evaluate the relevance of recommended solutions and provide

input based on their operational experiences would facilitate continuous improvement of the model, ensuring it evolves in accordance with real-world application and user expectations.

5.4 Entrepreneurship Idea

The TGS has strong potential for commercialization as a software solution for other organizations facing similar maintenance challenges. By offering it as a Software-as-a-Service (SaaS) product, it can be marketed to transportation companies, manufacturing industries, and technical service providers who require efficient maintenance support tools.

The commercial version could feature additional functions such as user management, customizable interfaces, system integration, and analytics dashboards. These enhancements would increase its value and usability across various sectors. Subscription-based models can be introduced, offering flexibility for businesses of different sizes and needs.

This entrepreneurial approach promotes innovation and encourages the development of local solutions to industrial challenges. It also reflects a sense of ownership and accountability, as well as a proactive mindset in addressing operational problems. By maintaining a strong feedback loop with users and continuously improving the product, the system can evolve into a reliable and impactful tool that contributes to safer and more efficient operations. Moreover, it demonstrates a responsible and forward-thinking attitude aligned with Prasarana's commitment to excellence and continuous improvement.

REFERENCES

- An, K., Yang, F., Lu, J., Li, L., Ren, Z., Huang, H., Wang, L., Zhao, P., Kang, Y., Ding, H., Lin, Q., Rajmohan, S., Zhang, D., & Zhang, Q. (2024). Nissist: An incident mitigation copilot based on troubleshooting guides. *Frontiers in Artificial Intelligence and Applications*. <https://doi.org/10.3233/faia241032>
- Anshuli Chikhale, Varshney, P., Monga, P., & Mehendale, N. (2024). Industrial defect detection model using computer vision. *SSRN Electronic Journal*. <https://doi.org/10.2139/ssrn.4814866>
- Charlton, A. (2023). *OCR report*. <https://era.ed.ac.uk/bitstream/handle/1842/40899/OCR%20Report%20Final.pdf?sequence=3>
- Chai, C. P. (2022). Comparison of text preprocessing methods. *Natural Language Engineering*, 29(3), 509–553. <https://doi.org/10.1017/S1351324922000213>
- Chan, R. (2020, September 30). The history of Malaysia’s rail system. *Gamuda Berhad*. <https://gamuda.com.my/2020/09/the-history-of-malaysia-rail-system/blog/>
- Choi, H., & Jeong, J. (2023). Realtime detection of table objects and text areas for OCR preprocessing. *WSEAS Transactions on Information Science and Applications*, 20, 197–205. <https://doi.org/10.37394/23209.2023.20.23>
- Elom, E. (2023). Implementation of a data aggregation ETL pipeline and business intelligence system. *CSCI 5910 Master’s Systems Development Project Final Report*. https://www.researchgate.net/profile/Emmanuel-Elom/publication/374914278_Data_Aggregation_ETL_Pipeline_and_Business_Intelligence_System/links/653642785d51a8012b64817c/Data-Aggregation-ETL-Pipeline-and-Business-Intelligence-System.pdf
- Elmira. (2022, February 24). Troubleshooting guide: Definition, importance, and tips for creating. *ClickHelp Technical Writing Blog*. <https://clickhelp.com/clickhelp-technical-writing-blog/troubleshooting-guide-definition-importance-and-tips-for-creating/>

- Glasenapp, L. A., Hoppe, A. F., Wisintainer, M. A., Sartori, A., & Stefenon, S. F. (2023). OCR applied for identification of vehicles with irregular documentation using IoT. *Electronics*, 12(5), 1083. <https://doi.org/10.3390/electronics12051083>
- G. B. Herwanto, Quirchmayr, G., & Tjoa, A. M. (2024). Leveraging NLP techniques for privacy requirements engineering in user stories. *IEEE Access*, 12(12), 1–1. <https://doi.org/10.1109/access.2024.3364533>
- IBM. (2024, June 14). What is a data pipeline. *IBM*. <https://www.ibm.com/topics/data-pipeline>
- IBM. (2024). What is digital transformation? *IBM*. <https://www.ibm.com/topics/digital-transformation>
- IBM Security Access Manager for Enterprise Single Sign-On 8.2.1. (n.d.). <https://www.ibm.com/docs/en/samfess/8.2.1?topic=troubleshooting-support>
- Li, Q., Kaneko, H., & Meng, L. (2025). A cosine similarity-based token subsampling method for vision transformer in cloud computing. *Neural Computing and Applications*, 37, 2627–2639. <https://doi.org/10.1007/s00521-024-10718-w>
- Lu, G., Ge, X., Zhong, T., Hu, Q., & Geng, J. (2024). Preprocessing enhanced image compression for machine vision. *IEEE Transactions on Circuits and Systems for Video Technology*, 1. <https://doi.org/10.1109/tcsvt.2024.3441049>
- Lu, X., Wang, L., Fan, S., Wang, X., & Houghton, J. (2023). ReadingQizMaker: A human-NLP collaborative system that supports instructors to design high-quality reading quiz questions. <https://doi.org/10.1145/3544548.3580957>
- Mao, D., & Wang, M. (2025). Encrypted search method for cloud computing data under attack based on TF-IDF and Apriori algorithm. *Applied Artificial Intelligence*, 39(1), e2449303. <https://doi.org/10.1080/08839514.2024.2449303>
- Monteiro, G., Camelo, L., Aquino, G., De A. Fernandes, R., Gomes, R., Printes, A., Torné, I., Silva, H., Oliveira, J., & Figueiredo, C. (2023). A comprehensive framework for industrial sticker information recognition using advanced OCR and object detection

- techniques. *Applied Sciences*, 13(12), 7320. <https://doi.org/10.3390/app13127320>
- Pandey, P. (2024, July 31). Analyze and optimize data pipelines with effective data models. *International Journal of Computer Trends and Technology*. <https://doi.org/10.14445/22312803/IJCTT-V72I7P111>
- Park, H., Lee, S., Gim, G., Kim, Y., Kim, D., & Park, C. (2024). Dataverse: Open-source ETL (extract, transform, load) pipeline for large language models. <https://doi.org/10.48550/arXiv.2403.19340>
- Peyton, K., & Unnikrishnan, S. (2023). A comparison of chatbot platforms with the state-of-the-art sentence BERT for answering online student FAQs. *Results in Engineering*, 17, 100856. <https://doi.org/10.1016/j.rineng.2022.100856>
- Putten, C. (2024, April). Transforming data flow: Generative AI in ETL pipeline automatization. *Webthesis Libraries*. <https://webthesis.biblio.polito.it/30855/>
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *arXiv*. <https://arxiv.org/abs/1908.10084>
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. <https://www.ukp.tu-darmstadt.de>
- Rinker, F., Waltersdofer, L., Meixner, K., Winkler, D., Luder, A., & Biffl, S. (2021). Continuous integration in multi-view modeling: A model transformation pipeline architecture for production systems engineering. *SCITEPRESS–Science and Technology Publications*. <https://pdfs.semanticscholar.org/4482/ae4531dcfc0dfc8a5154c5859cc48e62d639.pdf>
- Sadhvi. (2024, October 9). Data preprocessing in data mining: A hands-on guide. *Analytics Vidhya*. <https://www.analyticsvidhya.com/blog/2021/08/data-preprocessing-in-data-mining-a-hands-on-guide/>
- Schwab, K. (2018, May 25). The Fourth Industrial Revolution. *Encyclopedia Britannica*. <https://www.britannica.com/topic/The-Fourth-Industrial-Revolution-2119734>

- SentenceTransformers Documentation — Sentence Transformers documentation. (2019). *Sbert.net*. https://sbert.net/?utm_source=chatgpt.com
- Siino, M., Tinnirello, I., & La Cascia, M. (2023). Is text preprocessing still worth the time? A comparative survey on the influence of popular preprocessing methods on transformers and traditional classifiers. *Information Systems*, 121, 102342. <https://doi.org/10.1016/j.is.2023.102342>
- Sowmya Vajjala, Majumder, B., Gupta, A., & Surana, H. (2020). *Practical natural language processing: A comprehensive guide to building real-world NLP systems*. O'Reilly Media.
- Suleykin, A., & Panfilov, P. (2022). Railway communication network digital reporting system: A case study. *Semantic Scholar*. <https://www.semanticscholar.org/paper/940d21be3fb65b04e15eb6b6e4a256bb90347957>
- Tan, K., Wu, J., Zhou, H., Wang, Y., & Chen, J. (2024, January 25). Integrating advanced computer vision and AI algorithms for autonomous driving systems. [https://doi.org/10.53469/jtpes.2024.04\(01\).06](https://doi.org/10.53469/jtpes.2024.04(01).06)
- Tavares, R. A. (2024, October 15). Comparison of image preprocessing techniques for vehicle license plate recognition using OCR: Performance and accuracy evaluation. <https://doi.org/10.48550/arXiv.2410.13622>
- Tsang, S.-H. (2023, December 18). Brief review — Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *Medium*. <https://sh-tsang.medium.com/brief-review-sentence-bert-sentence-embeddings-using-siamese-bert-networks-afb473c2ff51>
- Tsvetkov, V., Shaytura, S., & Ordov, K. (2019). Digital management railway. *Advances in Economics, Business and Management Research*, 105, 201–204. <https://doi.org/10.2991/iscde-19.2019.34>
- Venkatesh, S. V., Akandeh, A., & Lokanath, M. (2024). MetaPix: A data-centric AI development platform for efficient management and utilization of unstructured computer vision data. *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.2409.12289>

APPENDICES

Appendix A: DSP Full Coding

<https://colab.research.google.com/drive/1M2NqrqxgEux03CLrWn1zlldgd3JxF5LU?usp=sharing>

Appendix B: Link app after Deployment

<https://app-tgs1git-m3zx35ugbsaxzsrzfe3yzg.streamlit.app/>